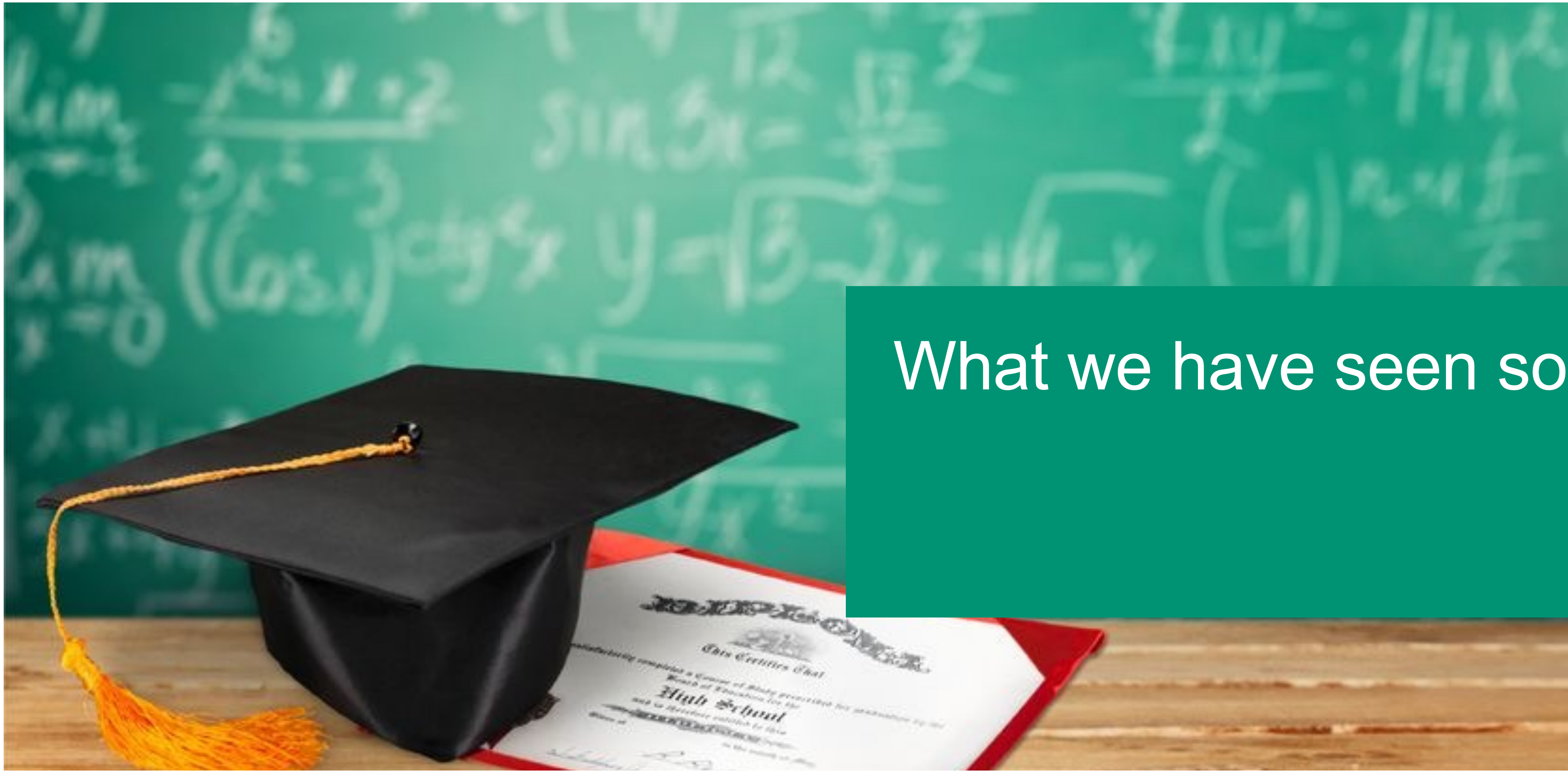
ETH zürich

Information Retrieval

8. Scoring in a bigger picture

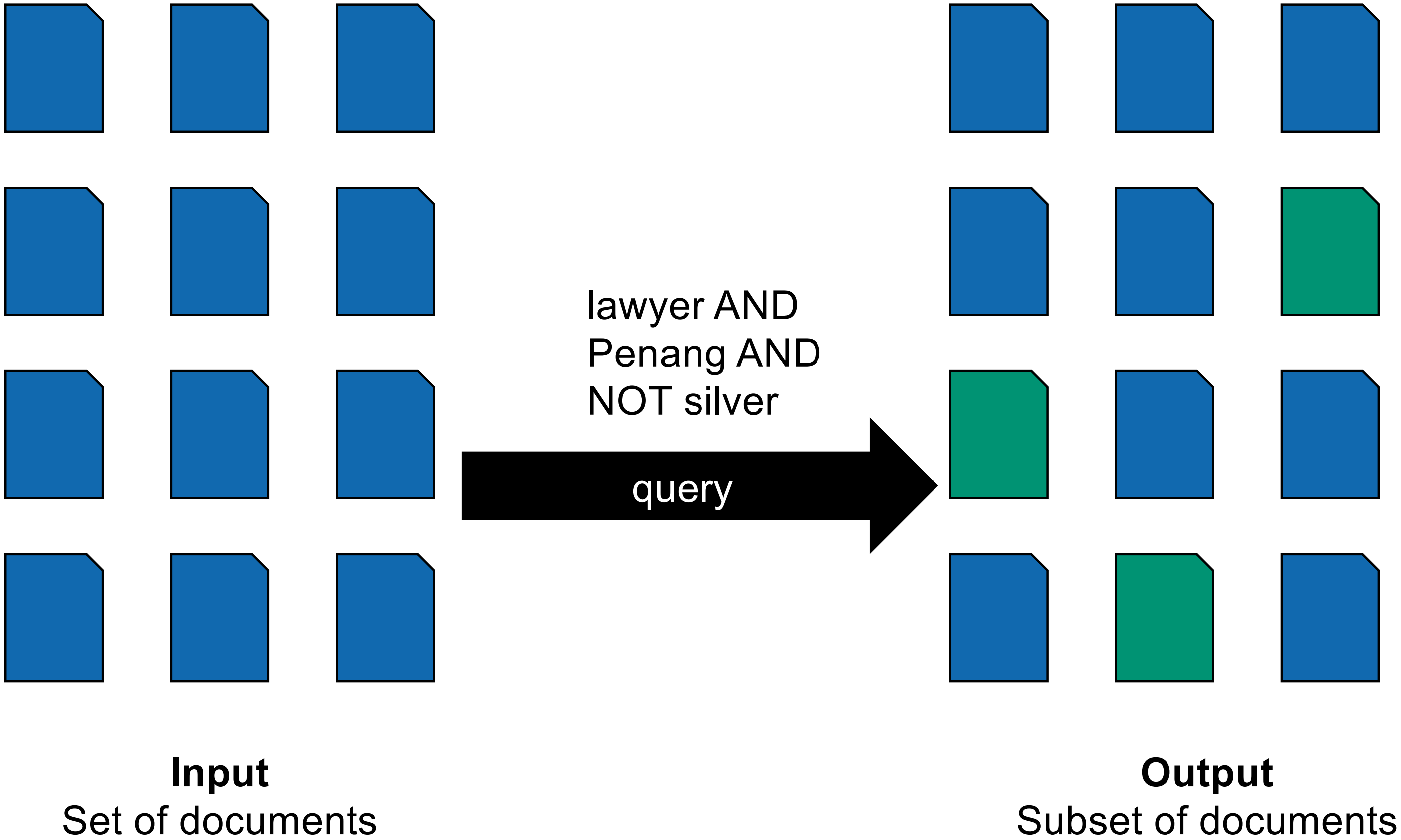
Ghislain Fourny
Spring 2022

Stock picture 123RF / fotoreactor

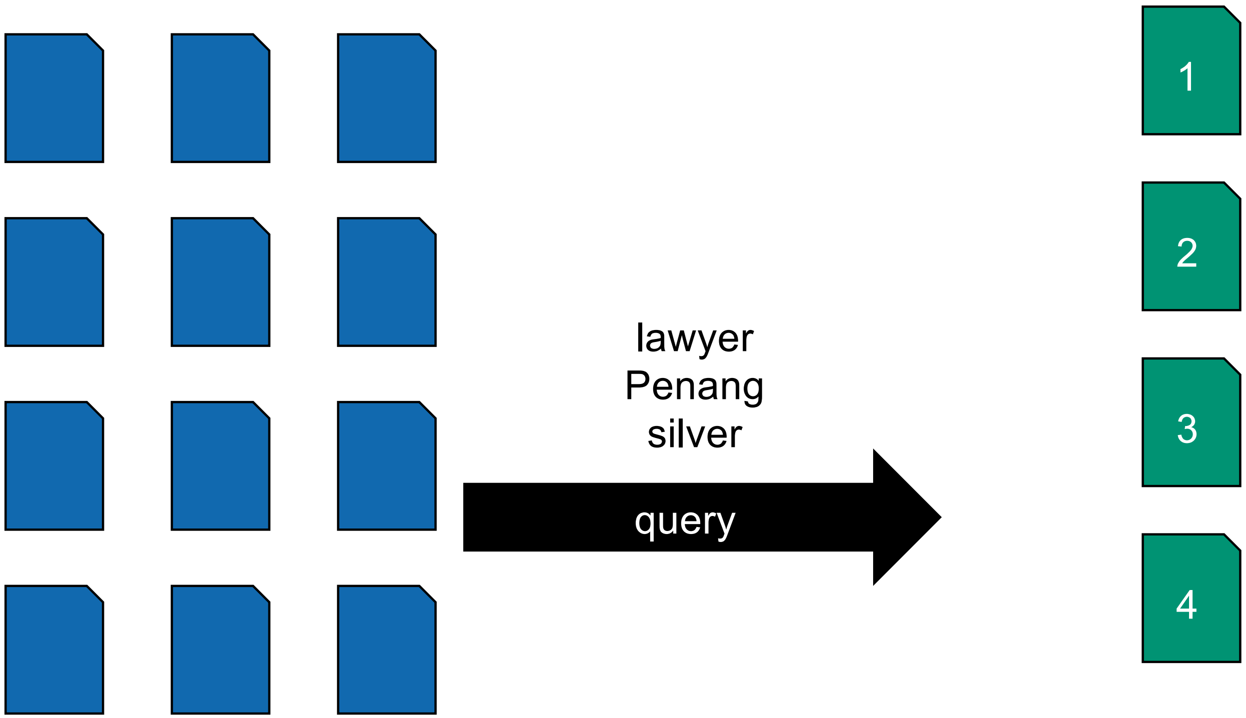


What we have seen so far

Boolean retrieval



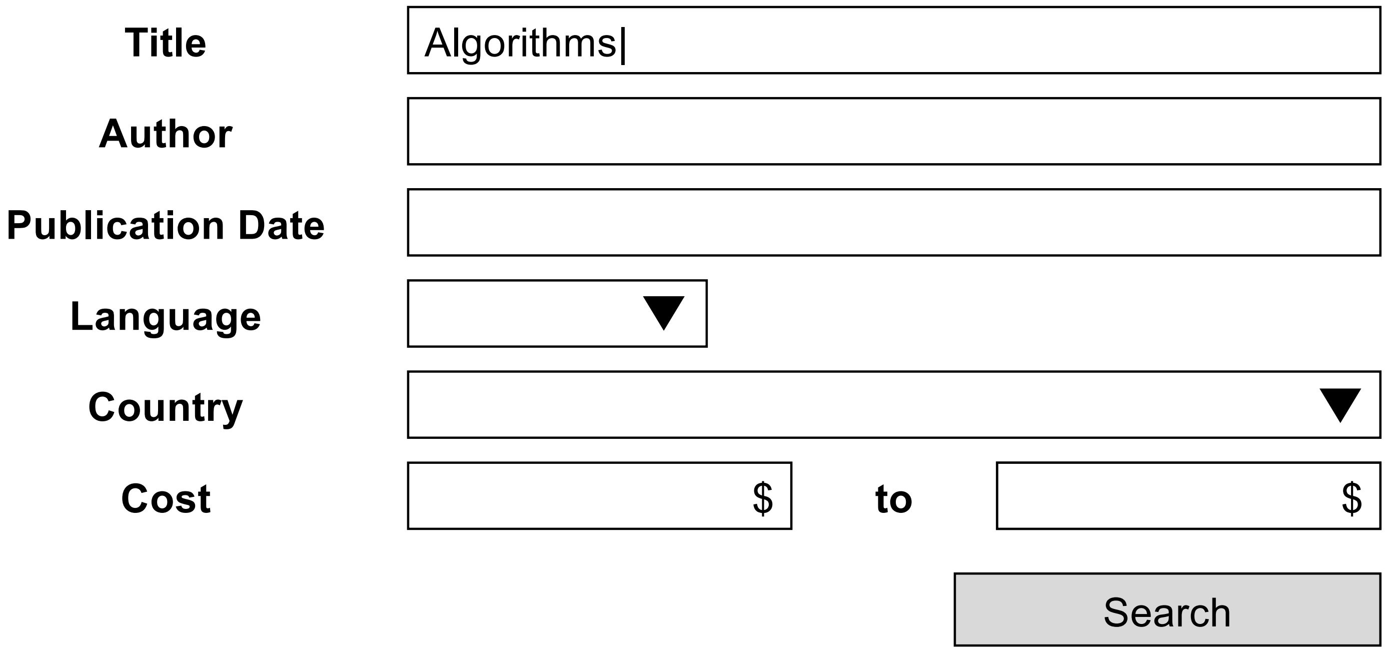
Today: Ranked retrieval



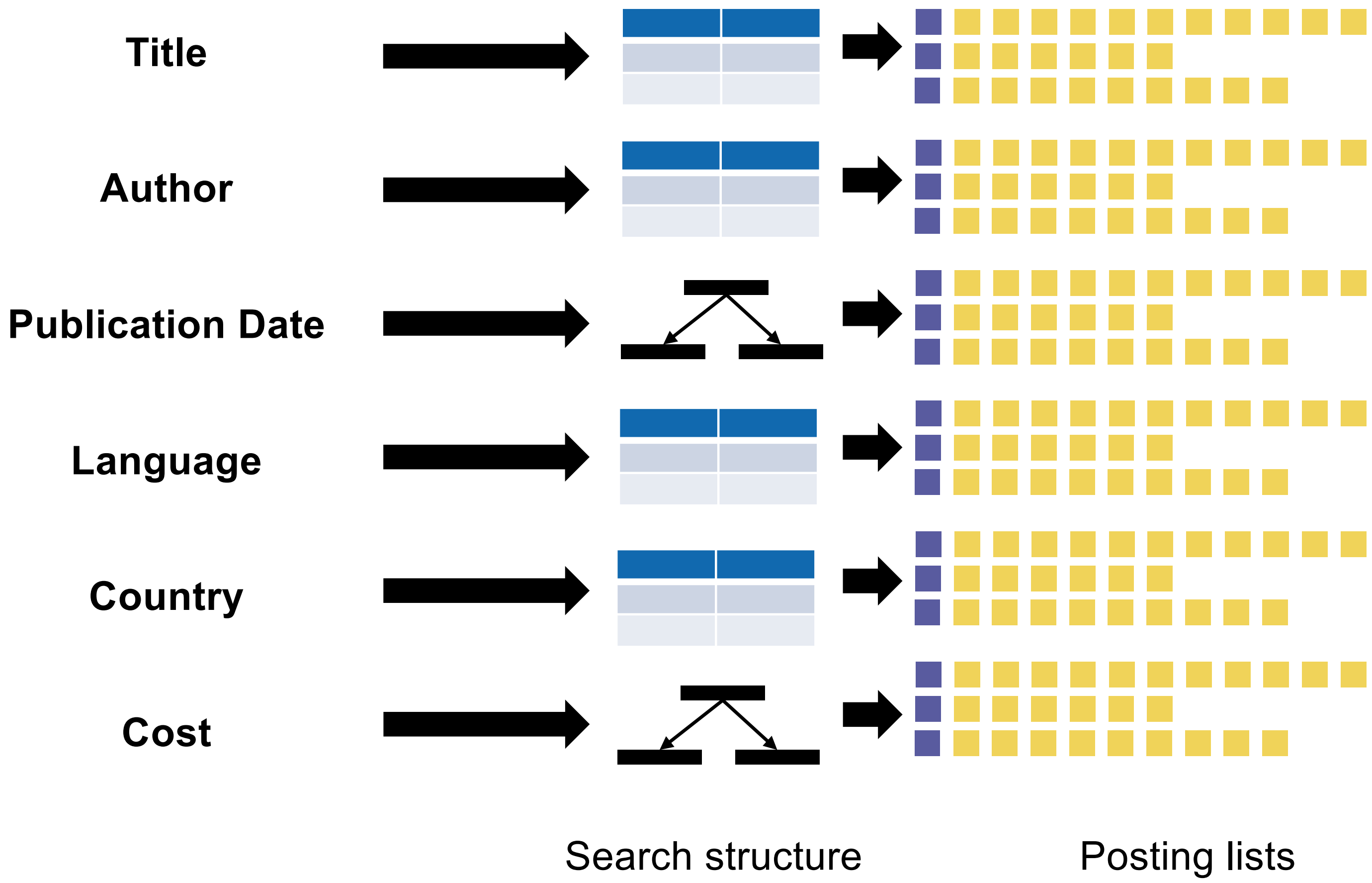
Input
Set of documents

Output
Ranked subset of documents

Parametric search



Parametric indices



Shared inverted index with zones

ETH	3	1.title	2.abstract	3.body
Zürich	2	1.title	3.title, 3.abstract	
computer	2	1.title, 1.abstract	4.body	
data	3	2.abstract	5.abstract	6.body
CPU	3	1.body	2.abstract	5.body
information	3	1.body	4.title, 4.body, 4.abstract	5.body
retrieval	3	1.body	6.body	7.title, 7.body, 7.abstract

Ranked retrieval with zones

information → 3 → 1.body | 4.title, 4.body | 5.body, 5.abstract

retrieval → 2 → 1.body | 4.title, 4.abstract

Zone	weight		information	retrieval	Total
Title	0.3	→	0.3	0.3	0.6
Abstract	0.2	→		0.2	0.2
Body	0.5	→	0.5		0.5

$$\vec{g}.\vec{s}$$

Total	1.3

Learning weights

$$\mathrm{argmin}_g \sum_j \mathrm{error}_j(g) = \mathrm{argmin}_g \sum_j (r_j - \vec{g_j}.\vec{s_j})^2$$

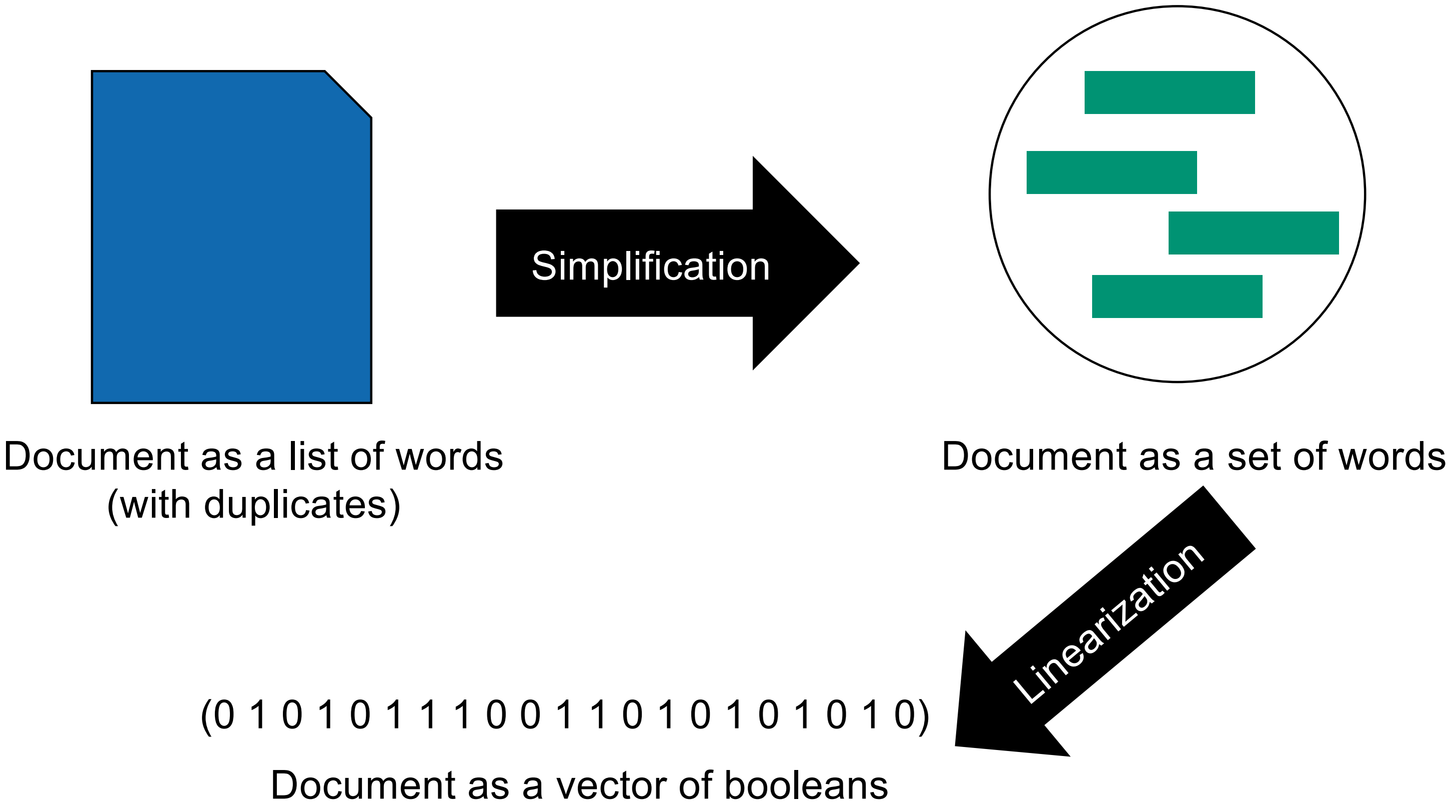
Term frequency, (Inverted) Document frequency,

tf	A	B
foo	5	1
bar	0	4
foobar	2	1

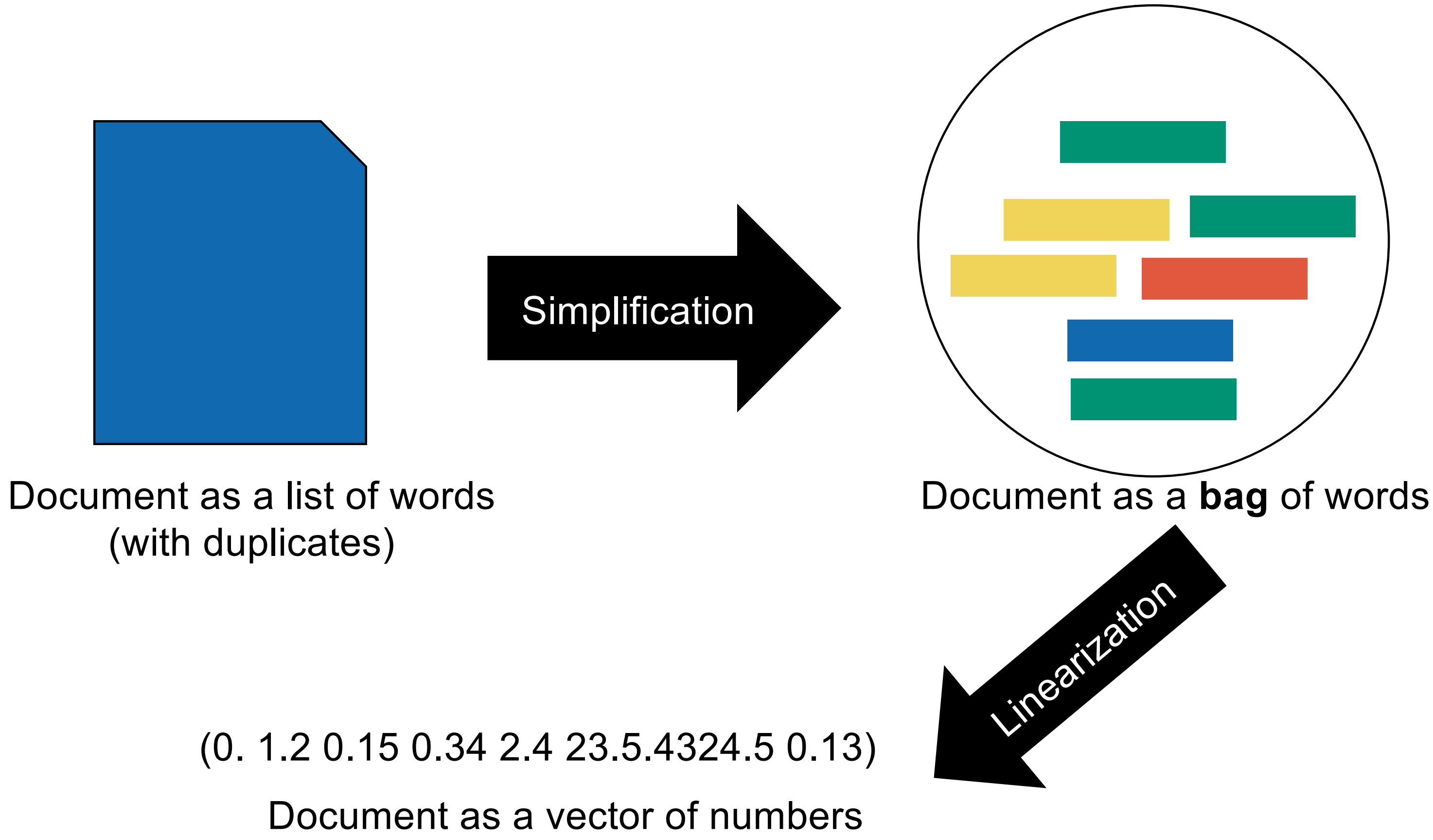
tf-idf	A	B
foo	25	5
bar	0	40
foobar	6	3

idf	
foo	5
bar	10
foobar	3

Model and abstraction

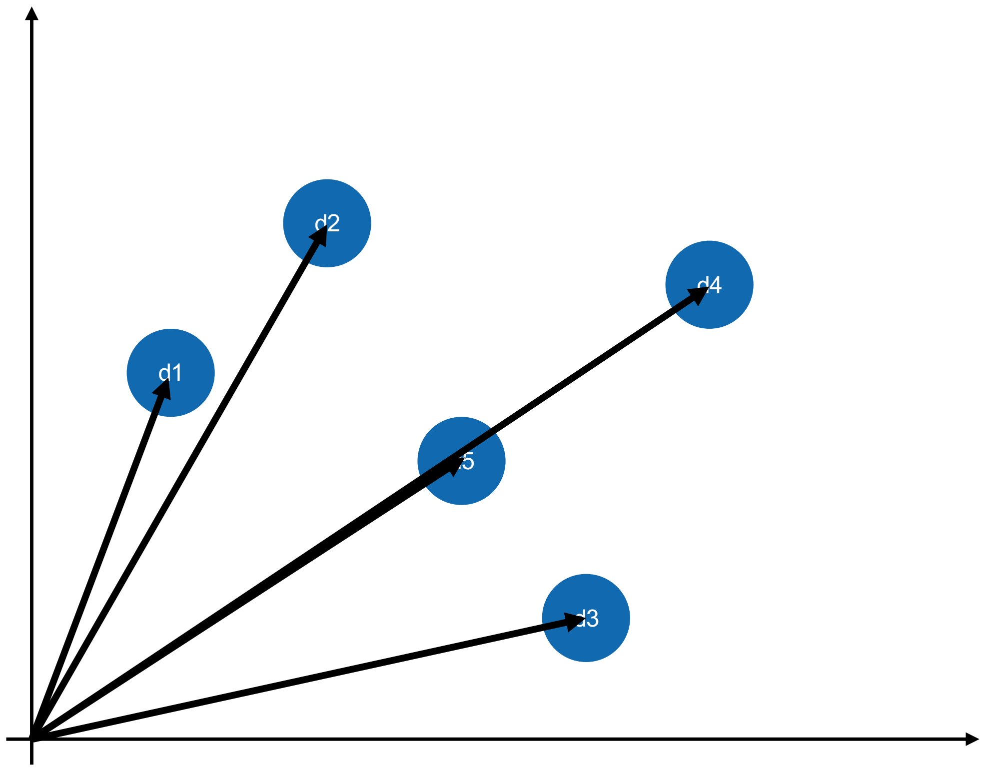


Model and abstraction



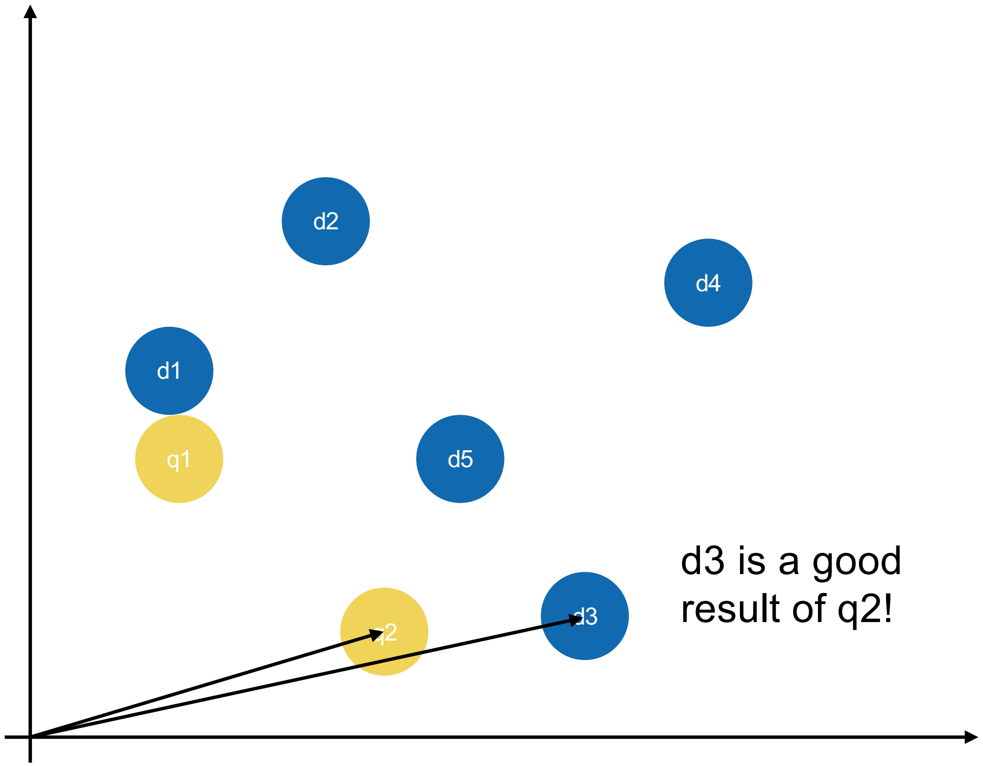
Vector-Space Model

Documents
= vectors in the
first quadrant
of
$\mathbb{R}^M$



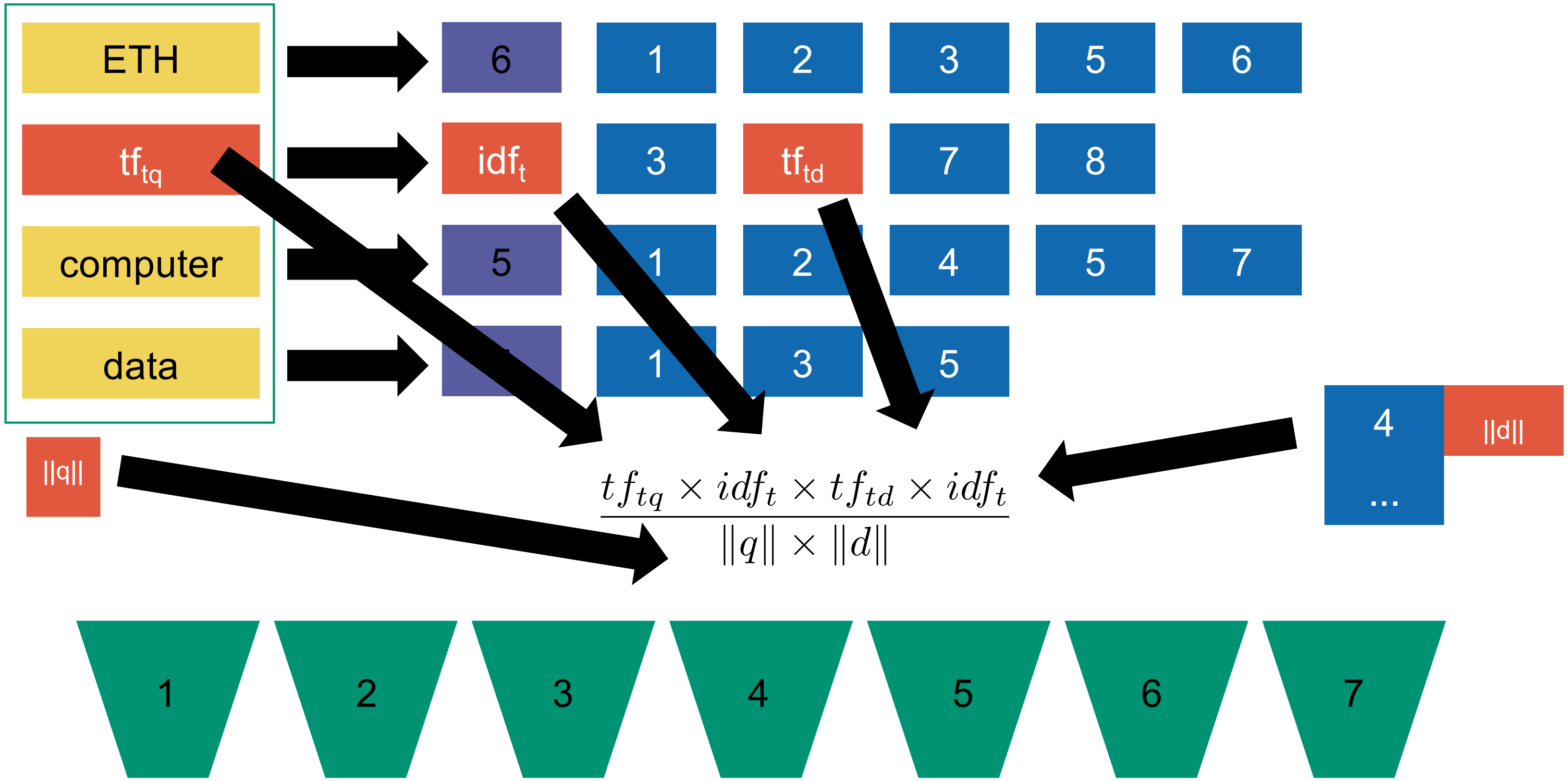
Queries as vectors

Queries
= points in the
first quadrant
of
$\mathbb{R}^M$



d3 is a good
result of q2!

Standard inverted index adapted to VSM



$$\frac{tf_{tq} \times idf_t \times tf_{td} \times idf_t}{\|q\| \times \|d\|}$$

SMART notation

Query weights

atc.lnb

Sublinear term frequency

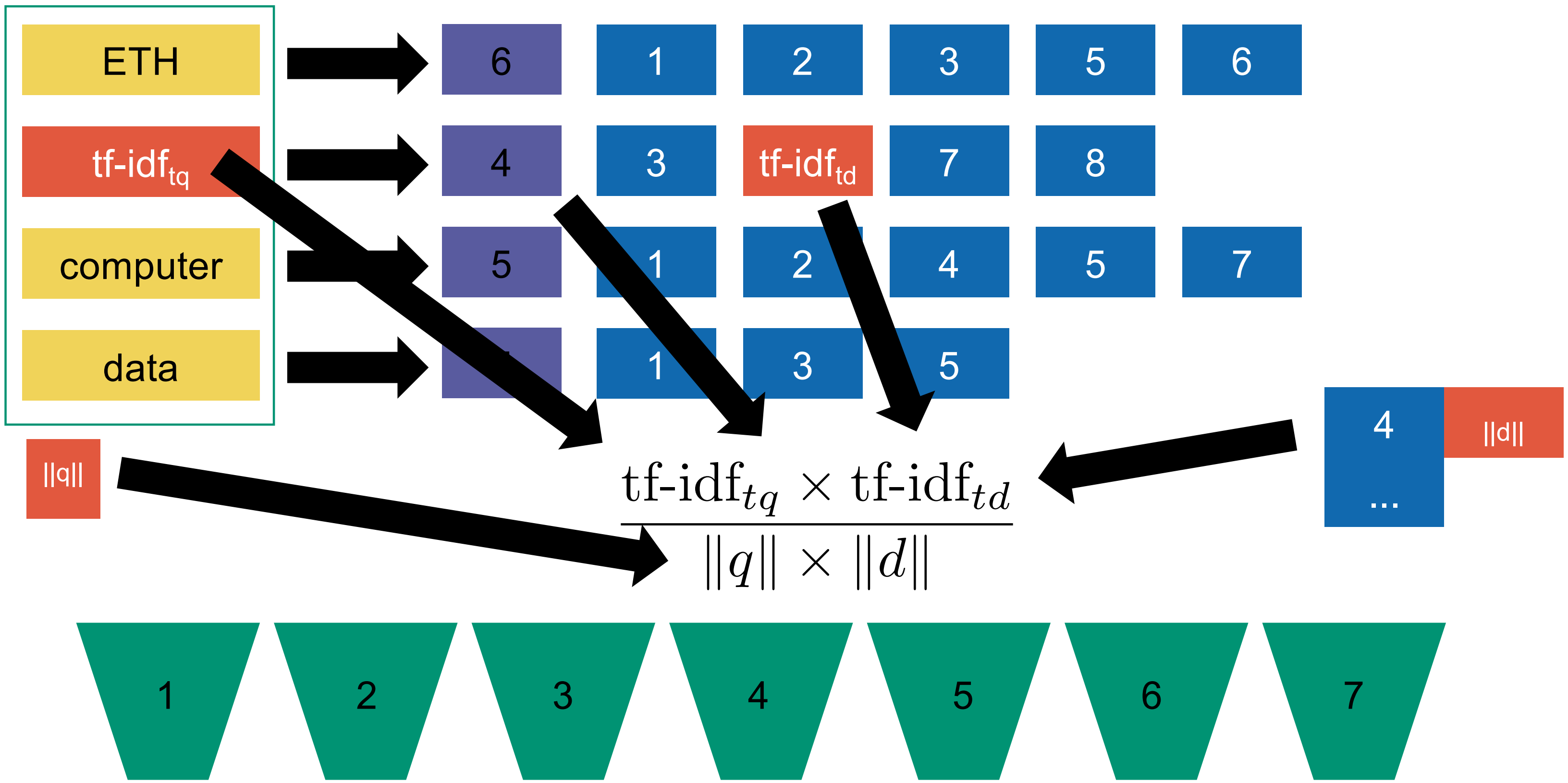
Byte-size normalization

Natural document frequency

ETH zürich

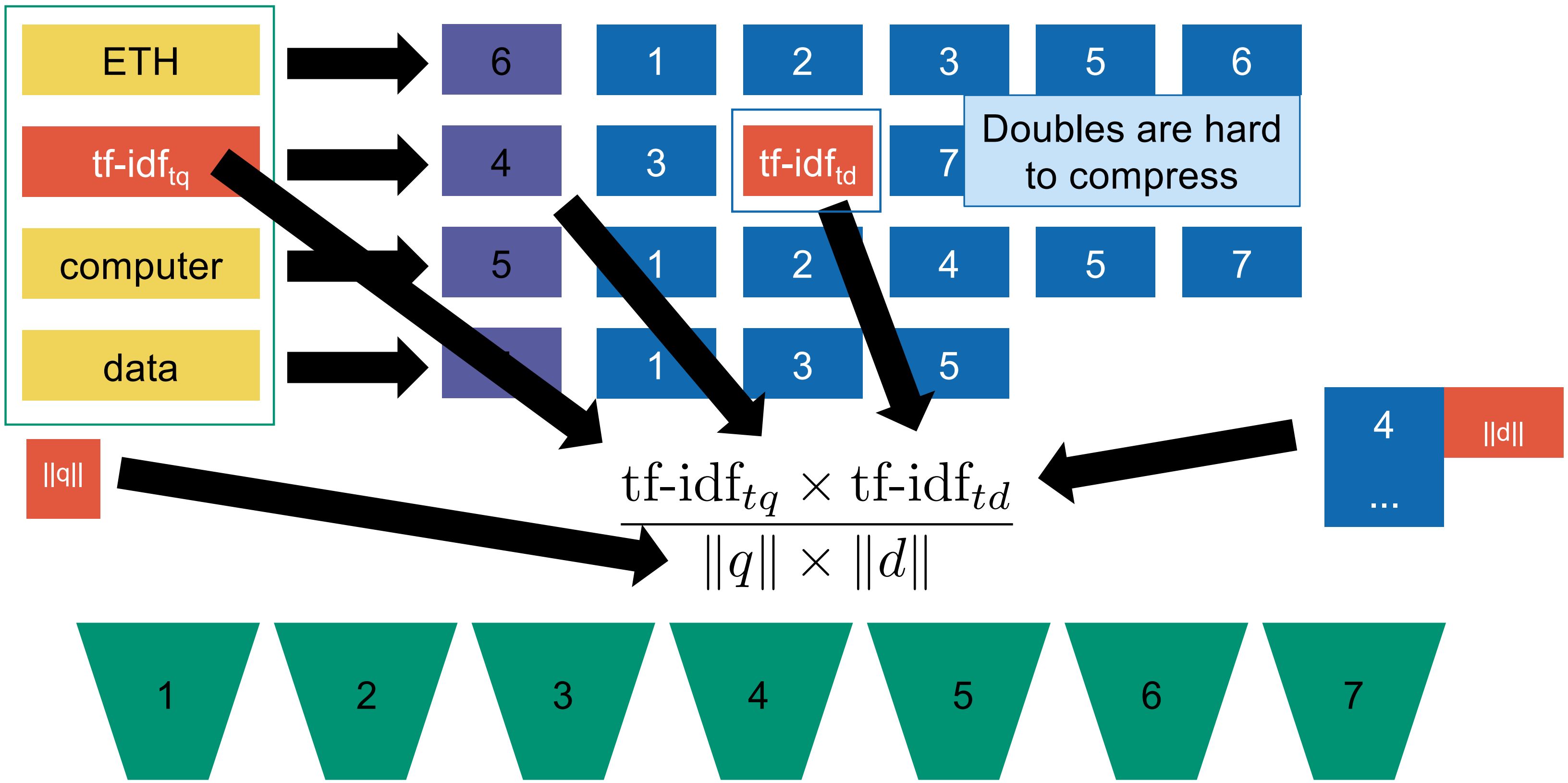
Efficient Scoring and Ranking

Standard inverted index adapted to VSM

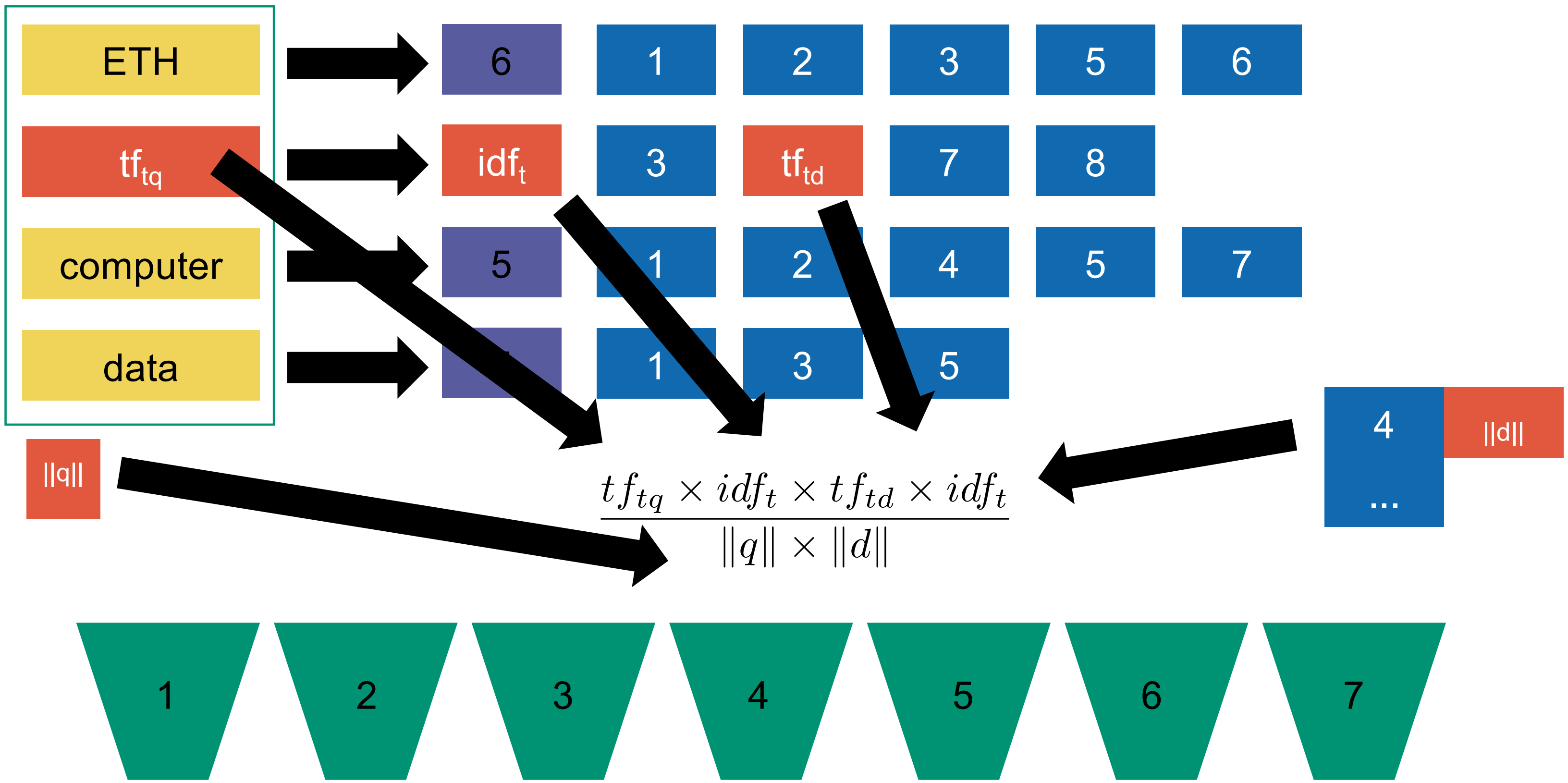


$$\frac{\text{tf-idf}_{tq} \times \text{tf-idf}_{td}}{\|q\| \times \|d\|}$$

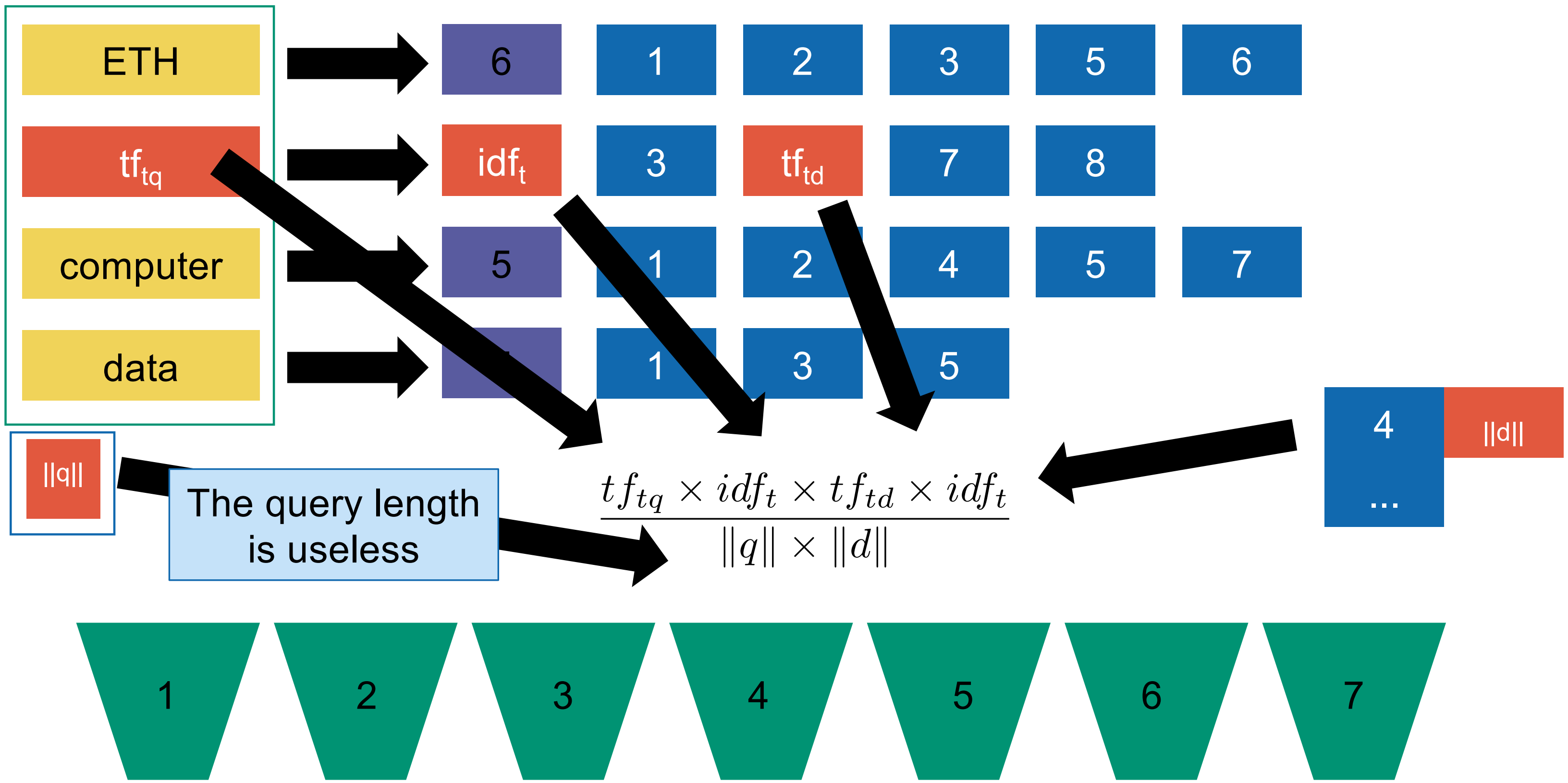
Standard inverted index adapted to VSM



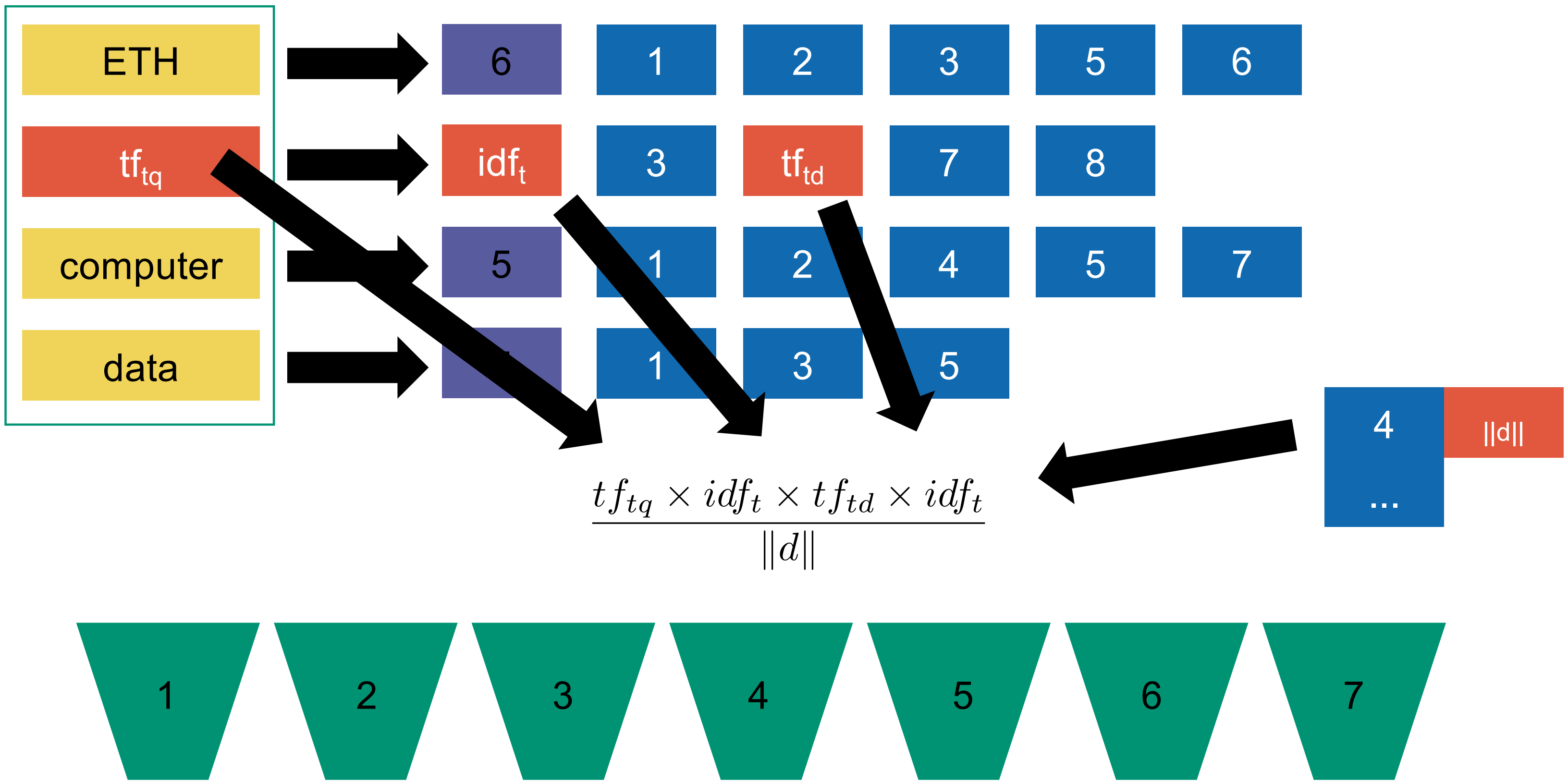
Standard inverted index adapted to VSM



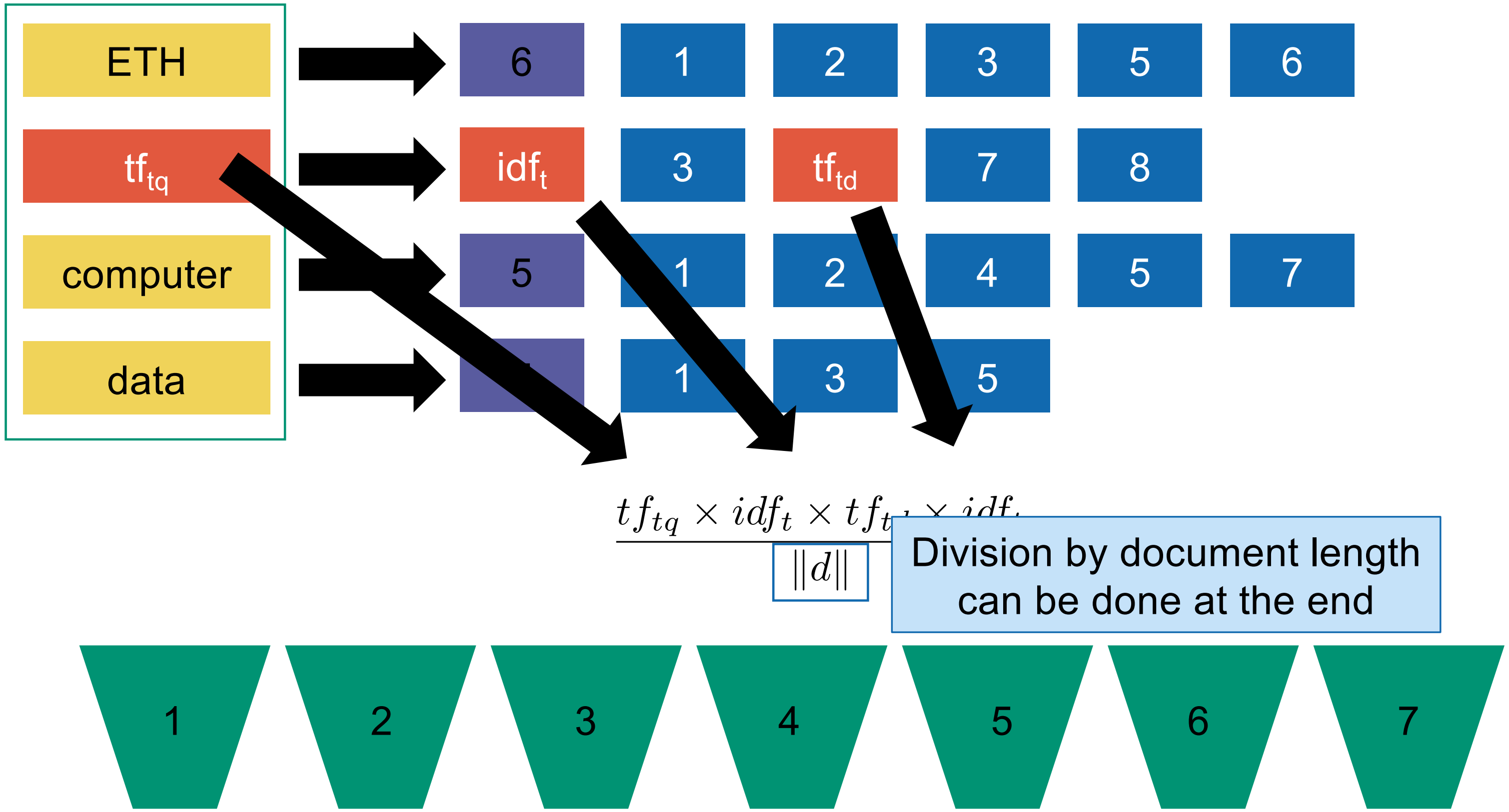
Standard inverted index adapted to VSM



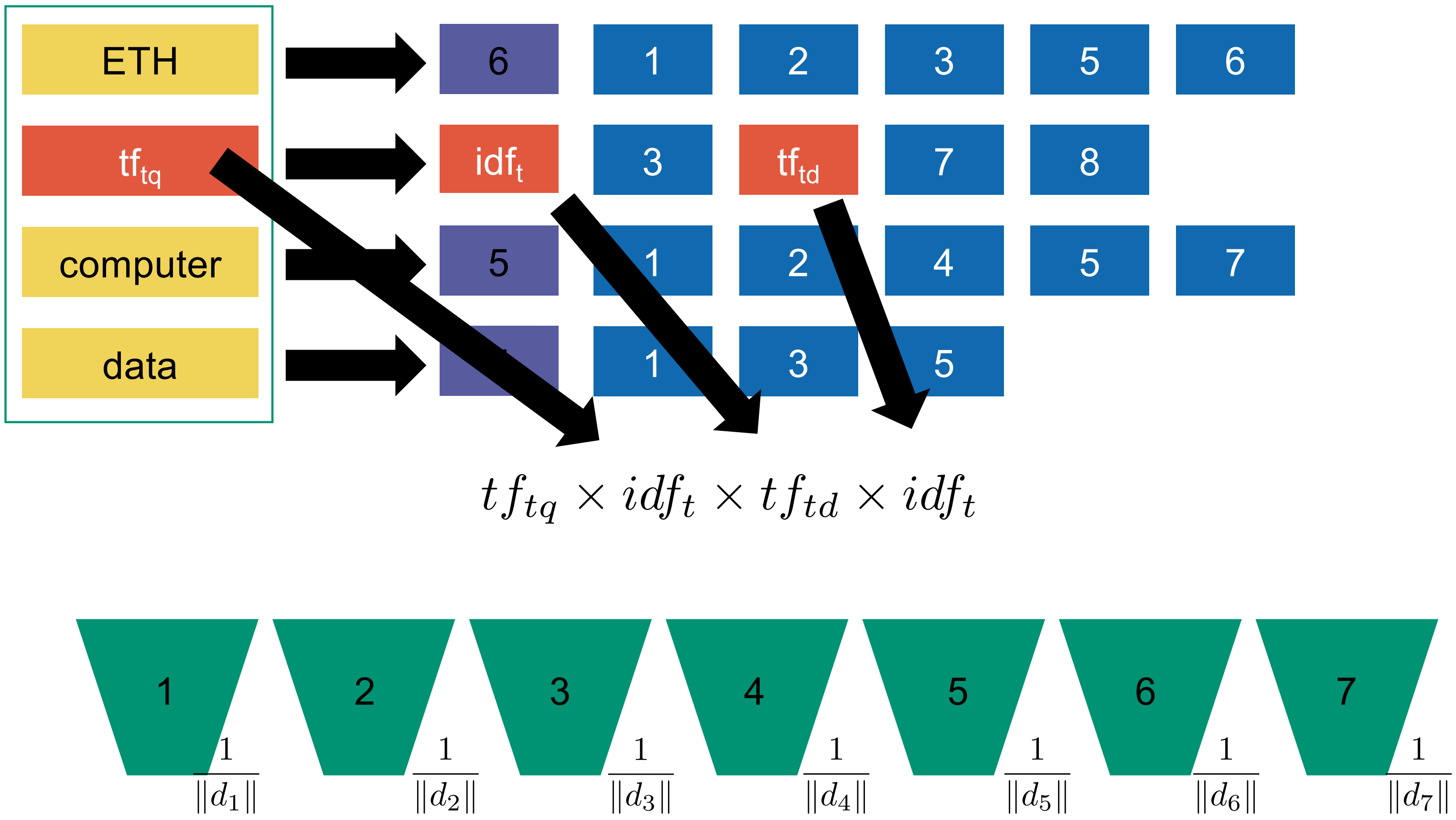
Standard inverted index adapted to VSM



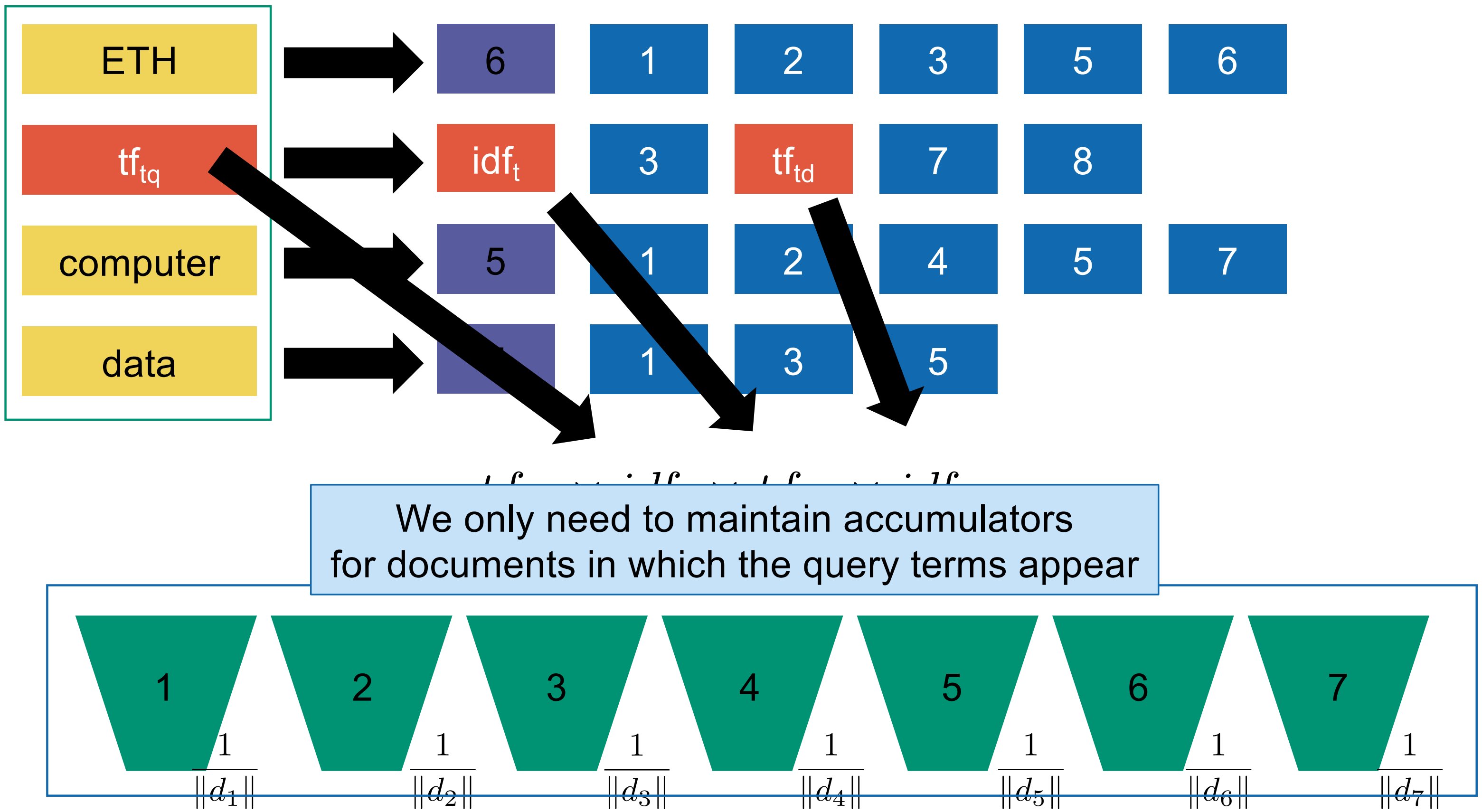
Standard inverted index adapted to VSM



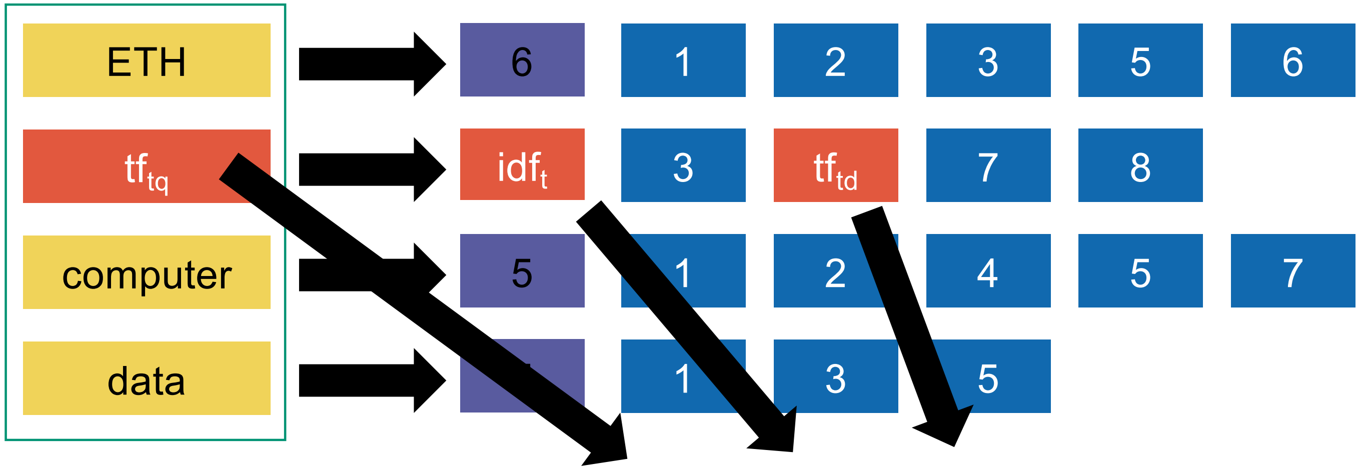
Standard inverted index adapted to VSM



Standard inverted index adapted to VSM



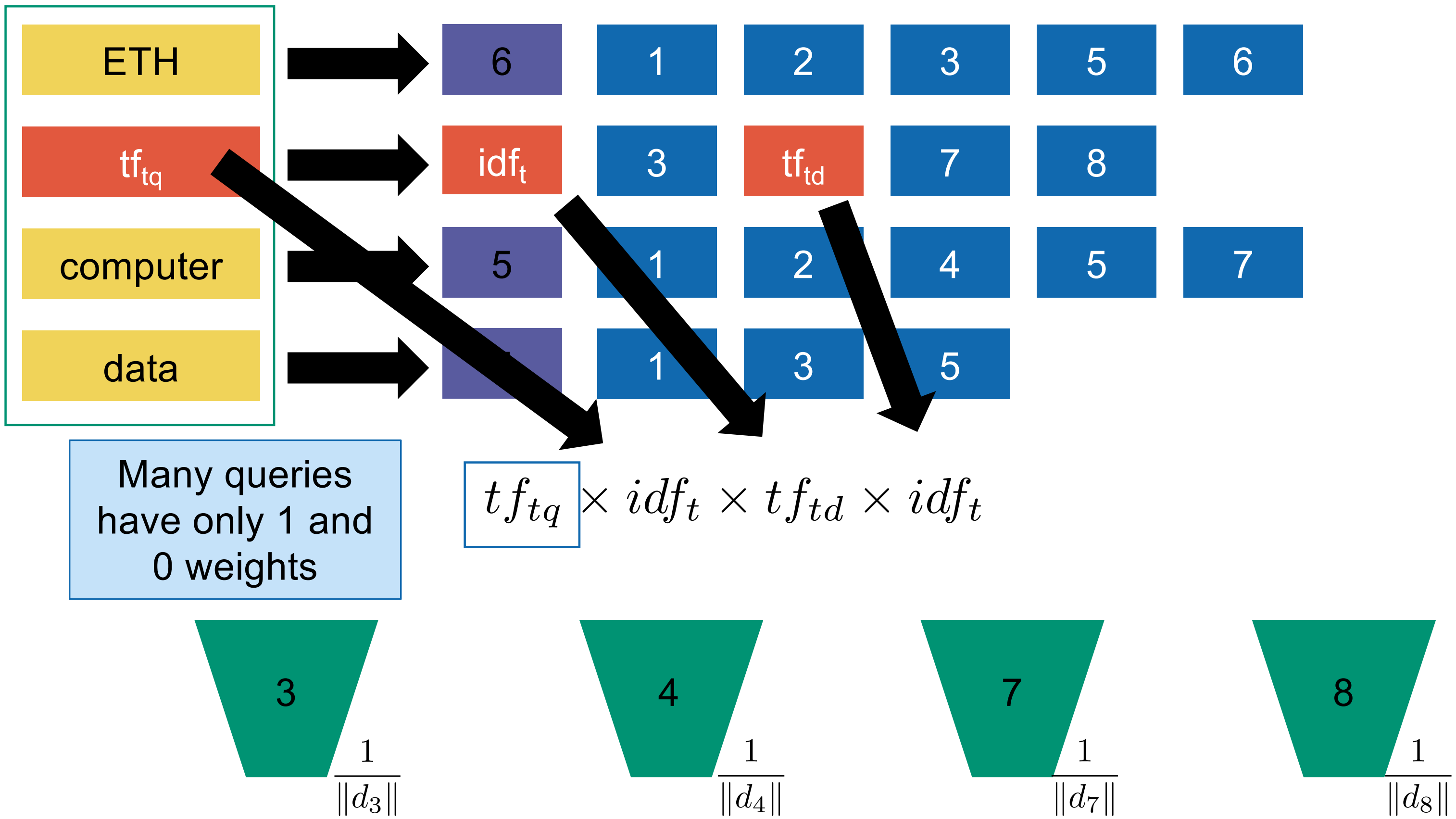
Standard inverted index adapted to VSM



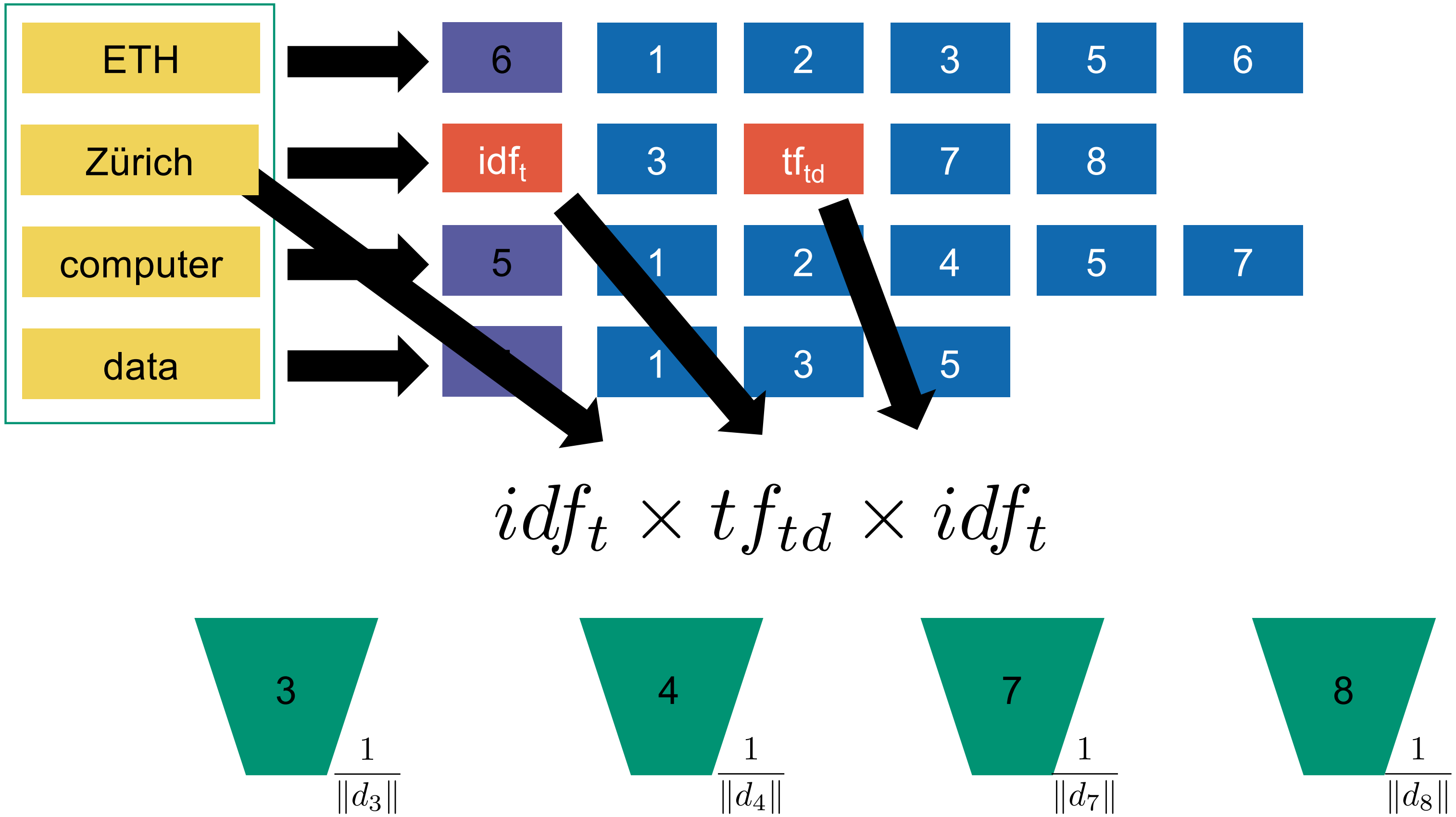
$$tf_{tq} \times idf_t \times tf_{td} \times idf_t$$

$\frac{1}{\|d_3\|}$ $\frac{1}{\|d_4\|}$ $\frac{1}{\|d_7\|}$ $\frac{1}{\|d_8\|}$

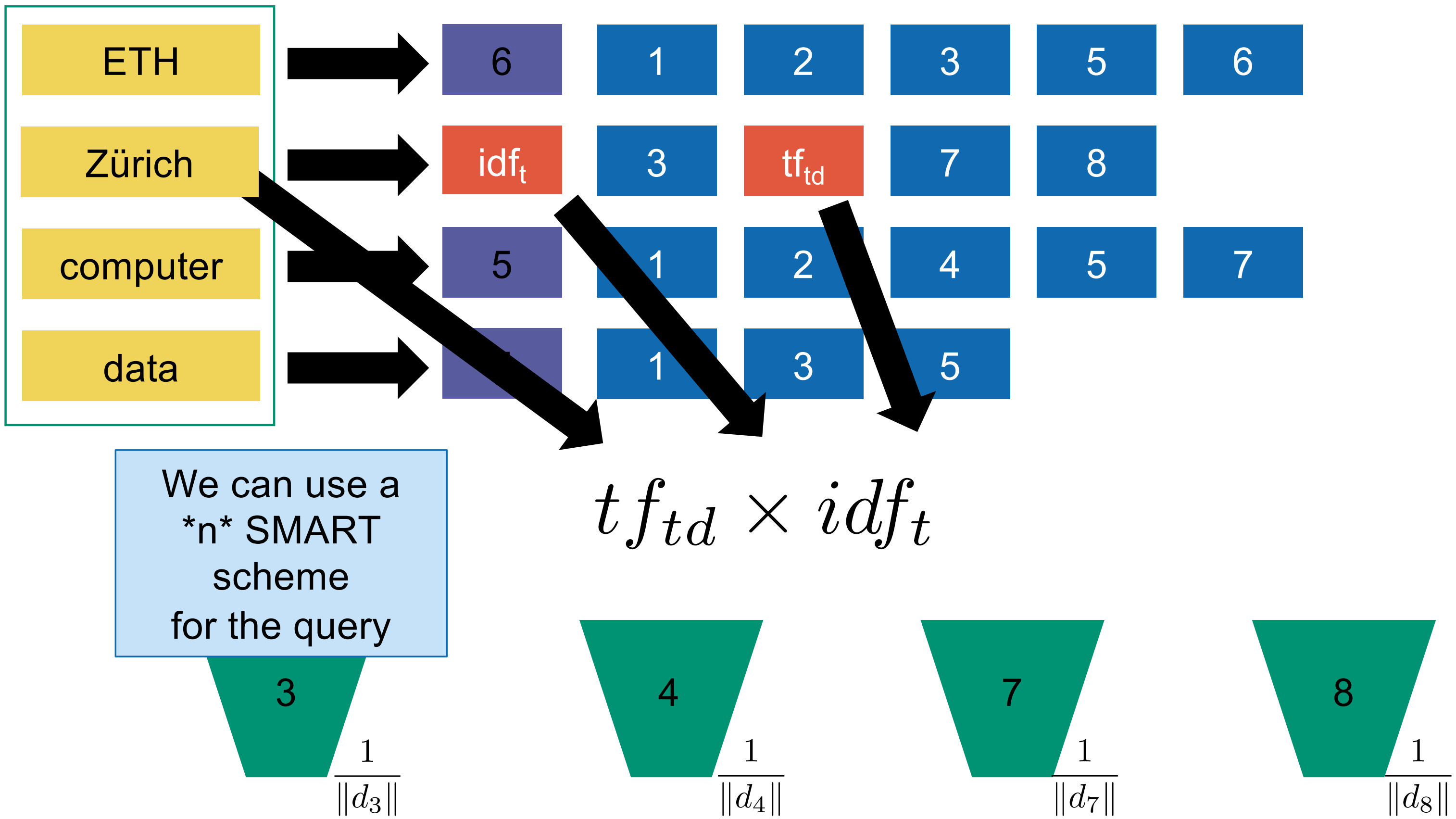
Standard inverted index adapted to VSM



Standard inverted index adapted to VSM



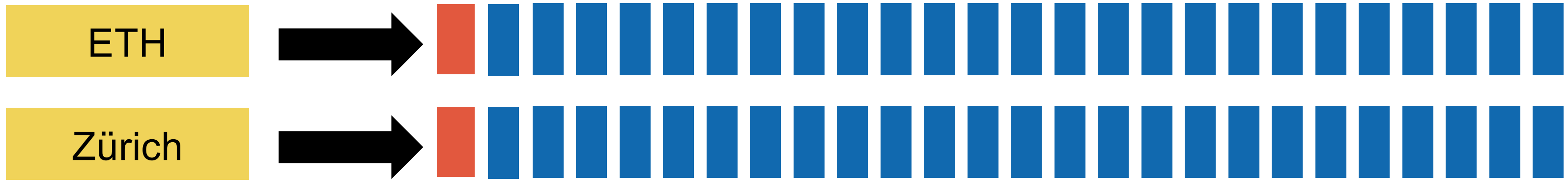
Standard inverted index adapted to VSM



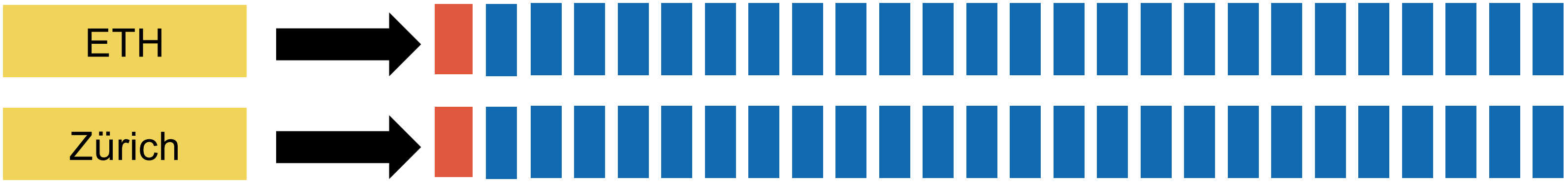
ETH zürich

Inexact top-K document retrieval

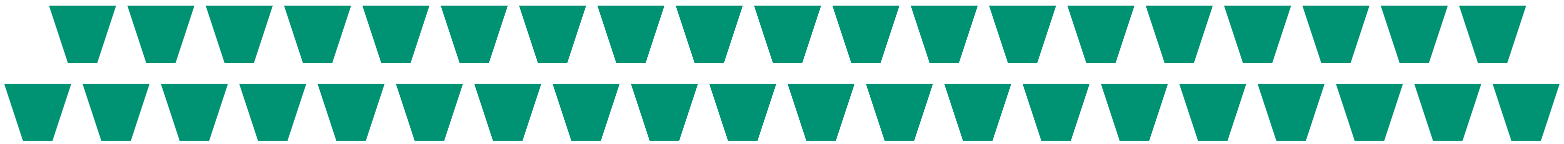
Posting lists can be very large!



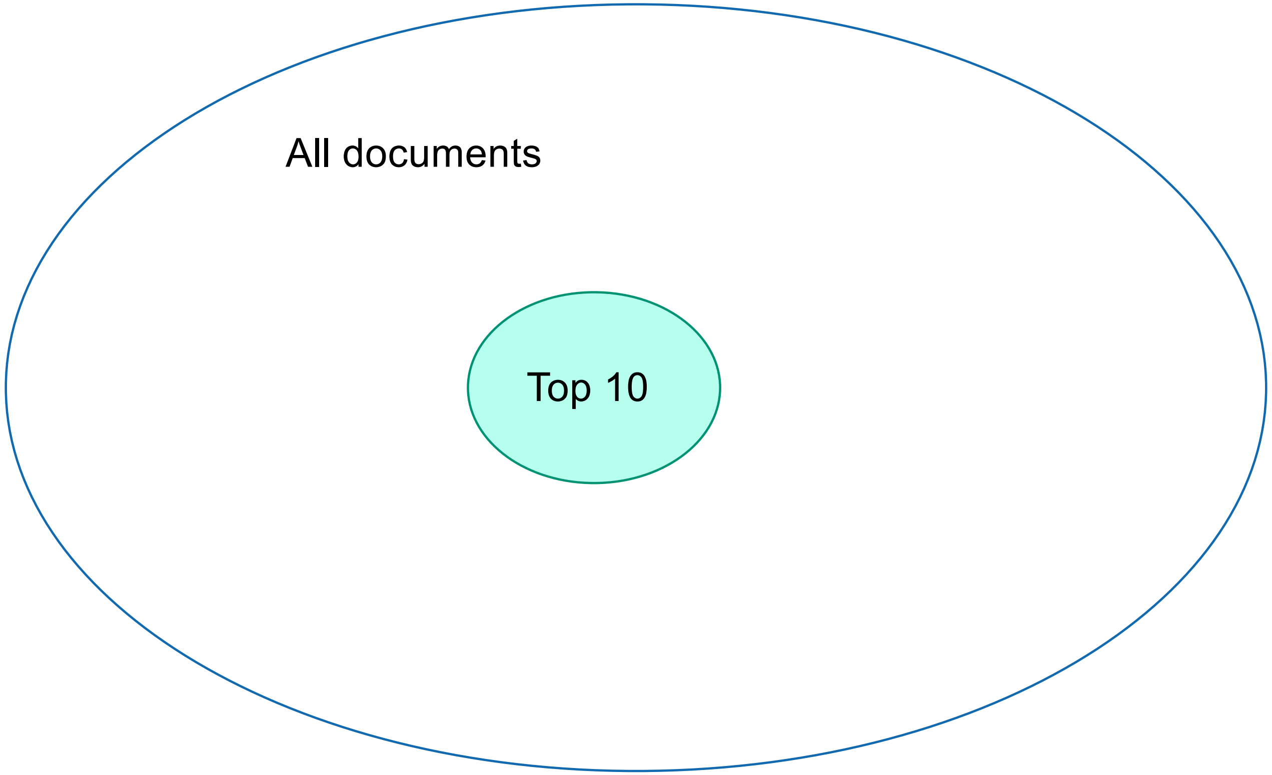
Posting lists can be very large!



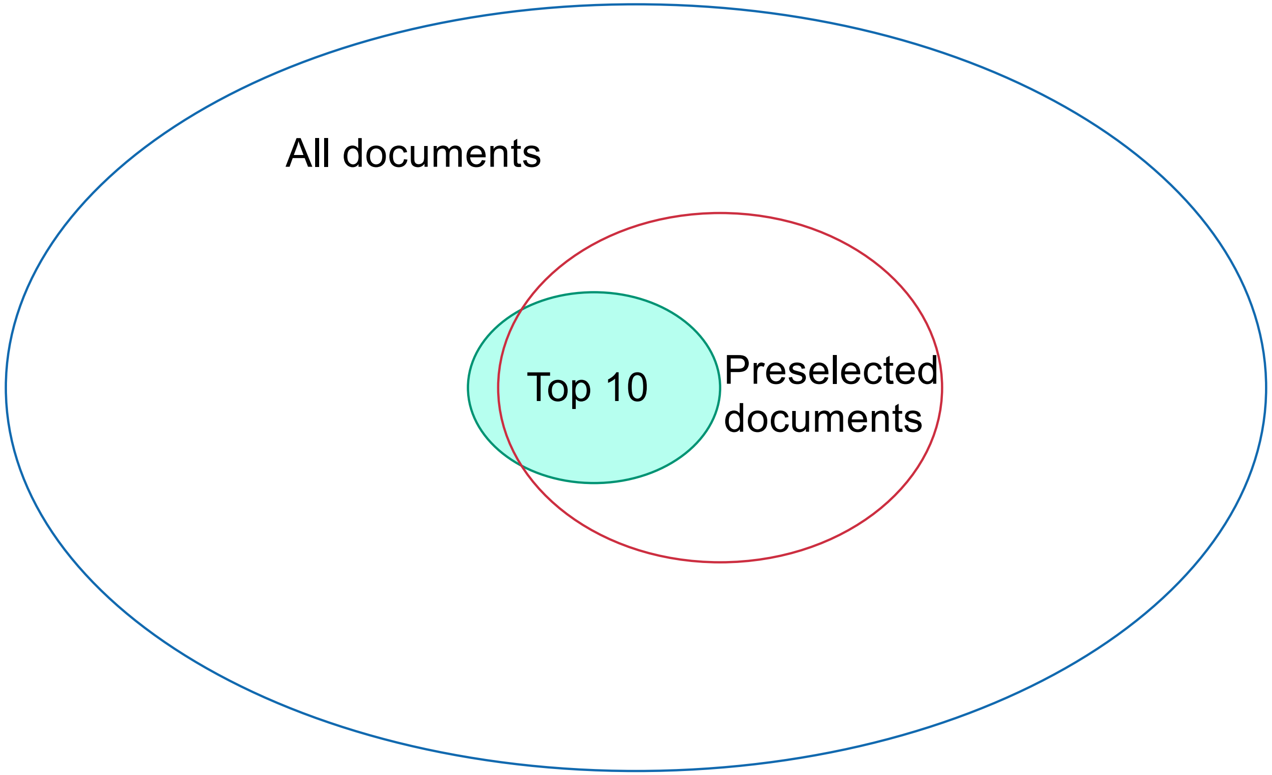
Many accumulators to maintain
Many computations



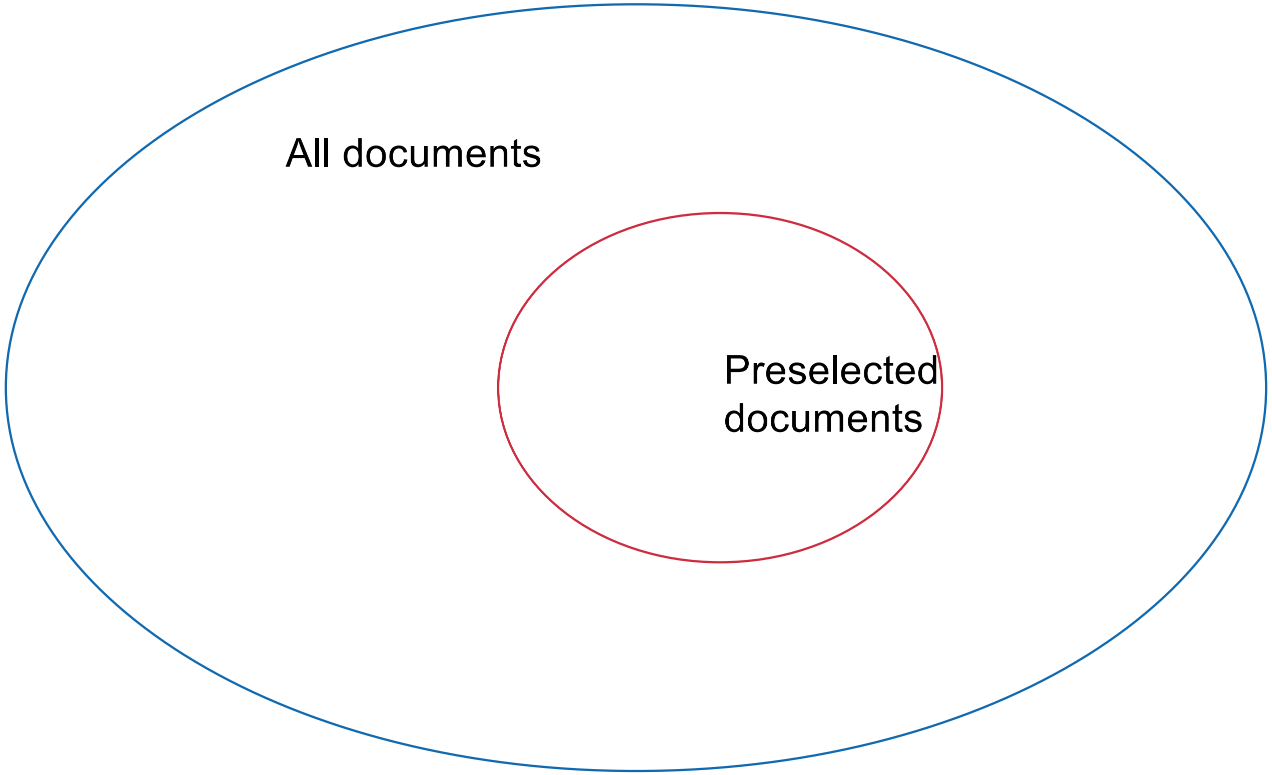
General optimization



General optimization



General optimization

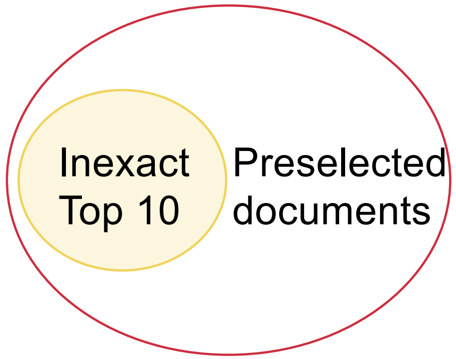


General optimization

Only compute scores in this smaller set!

General optimization

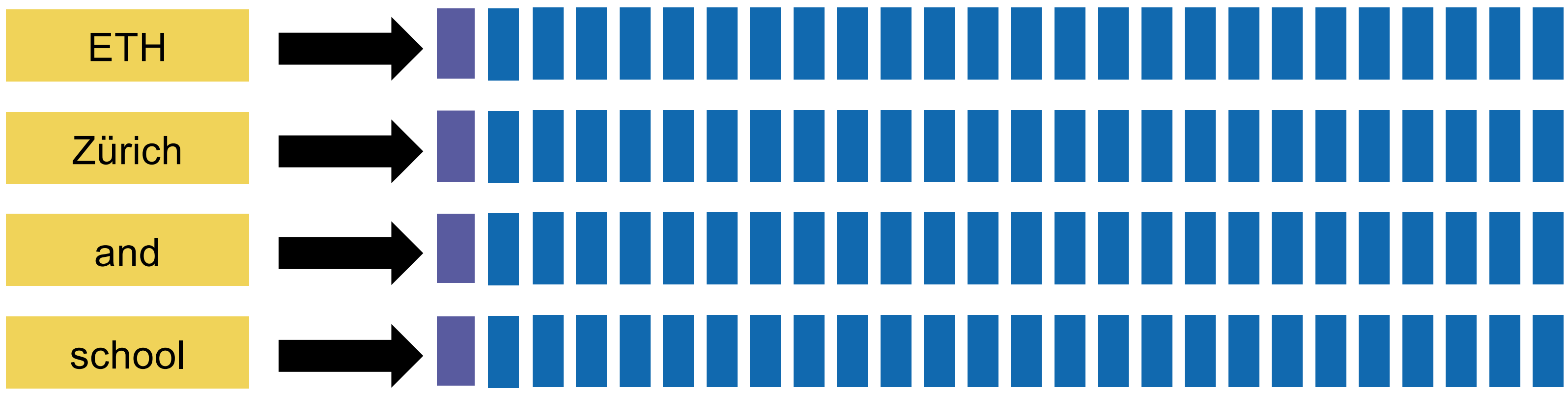
Only compute scores in this smaller set!



Index elimination

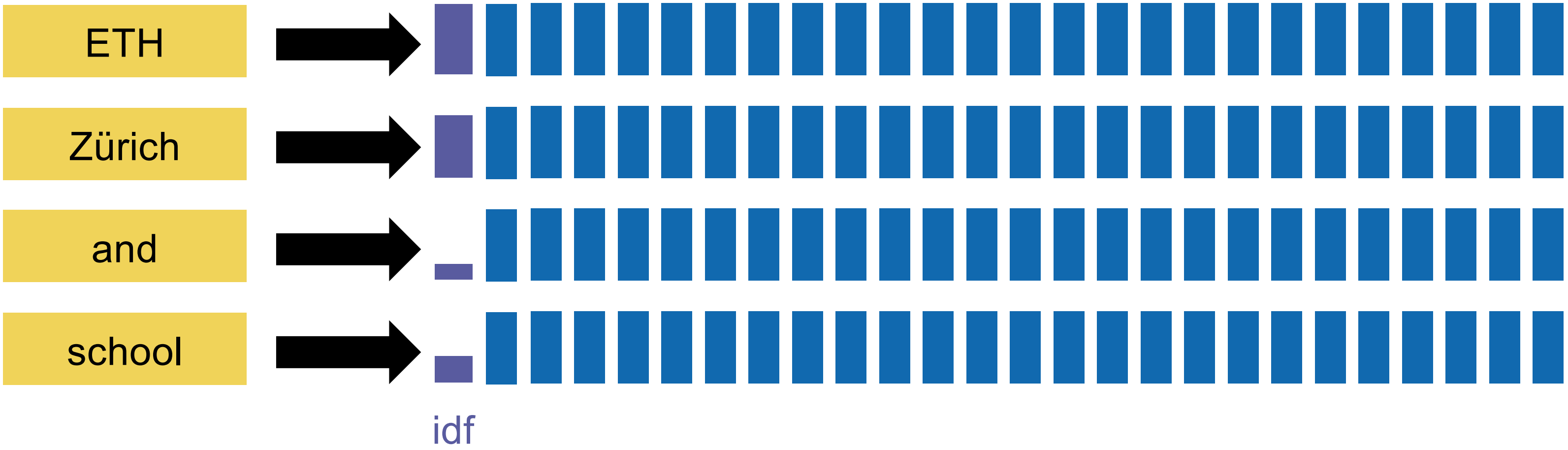
Query: `"ETH Zürich and school"`

Index elimination



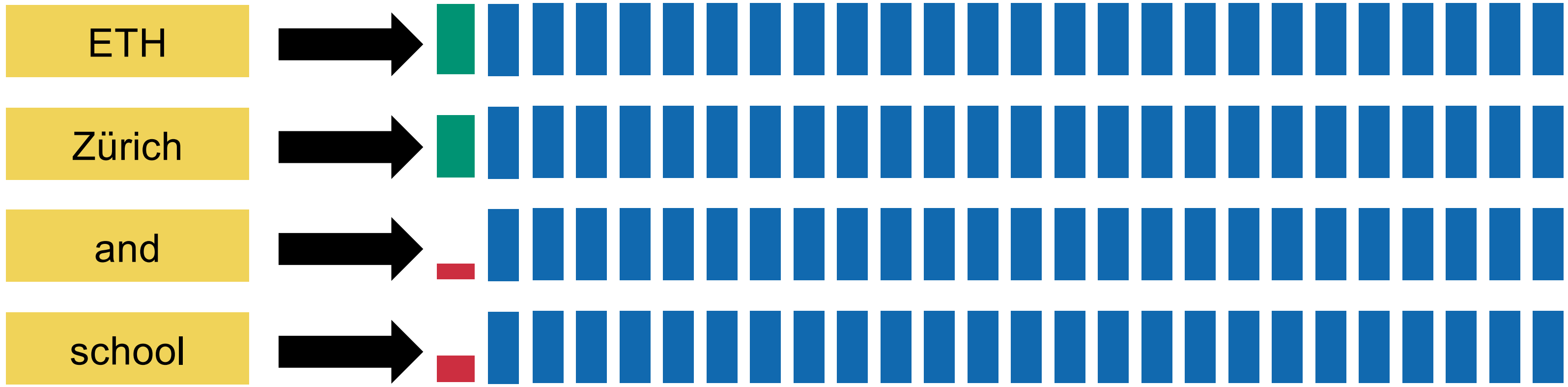
Query: `"ETH Zürich and school"`

Index elimination

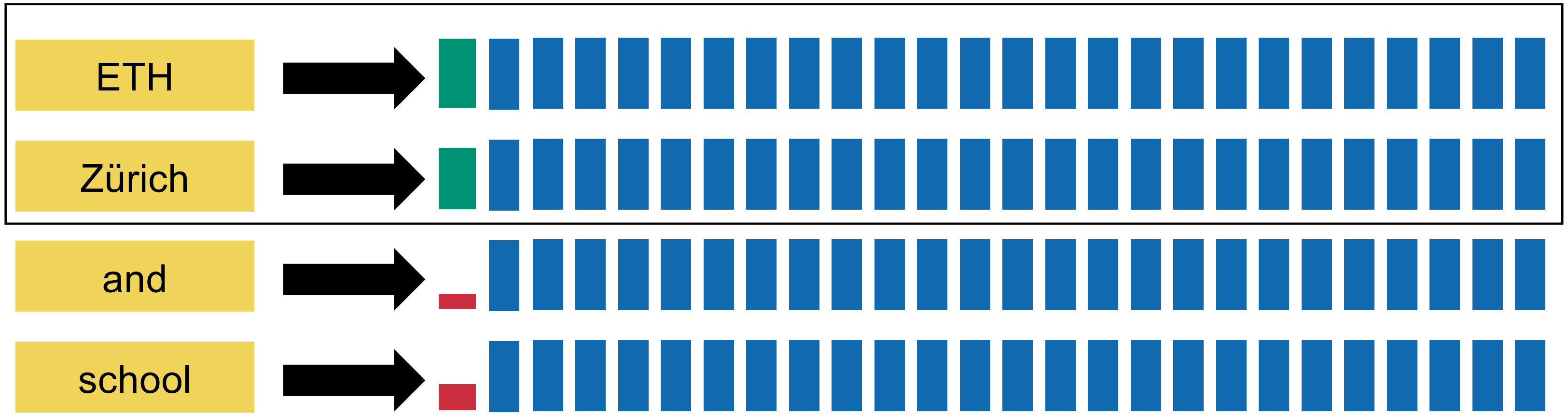


Query: `"ETH Zürich and school"`

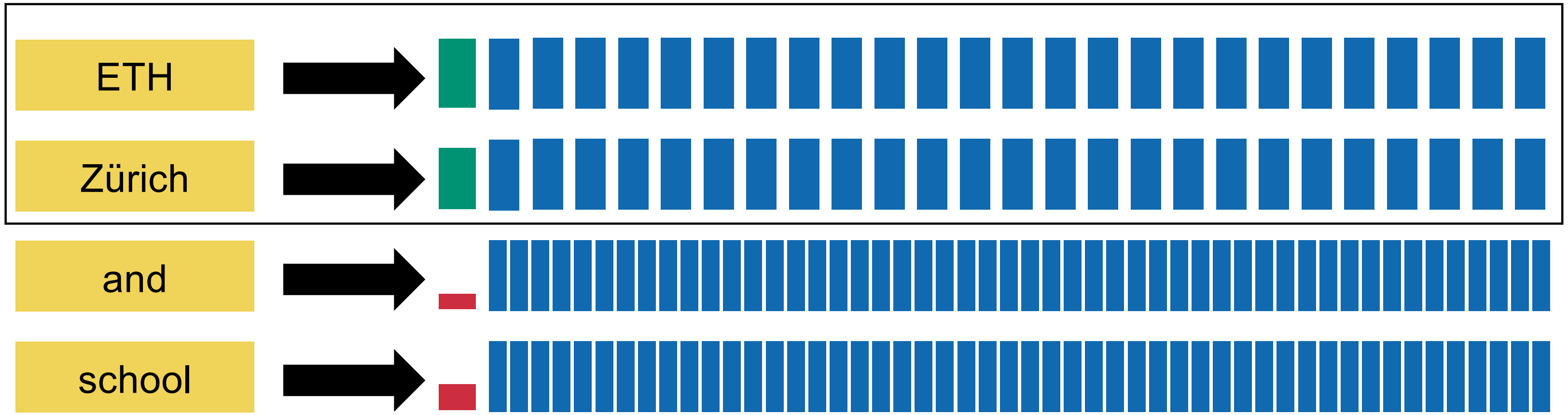
Index elimination: first idea



Index elimination: first idea

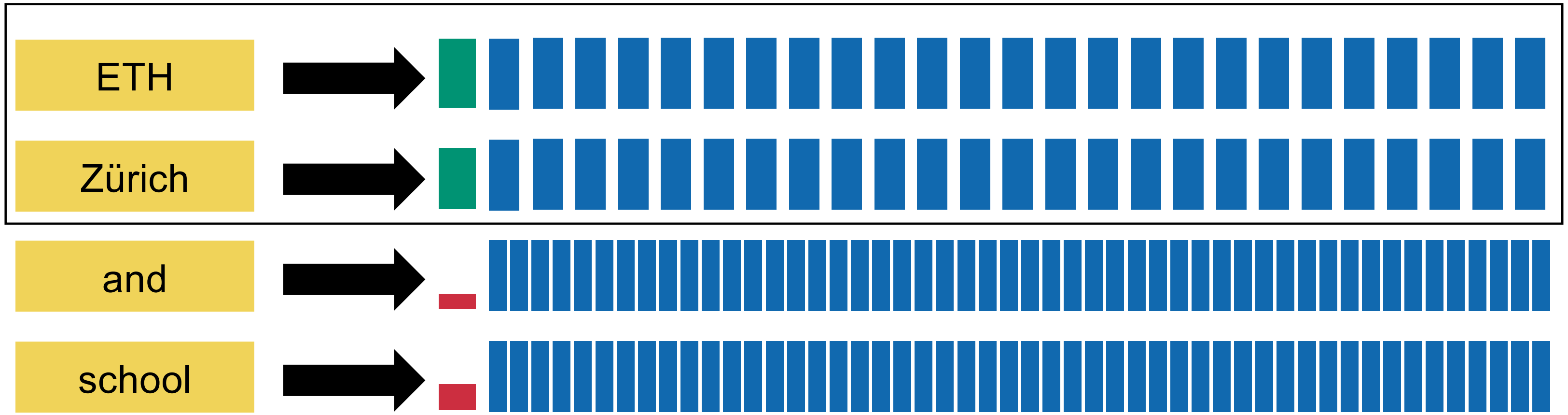


Index elimination: first idea



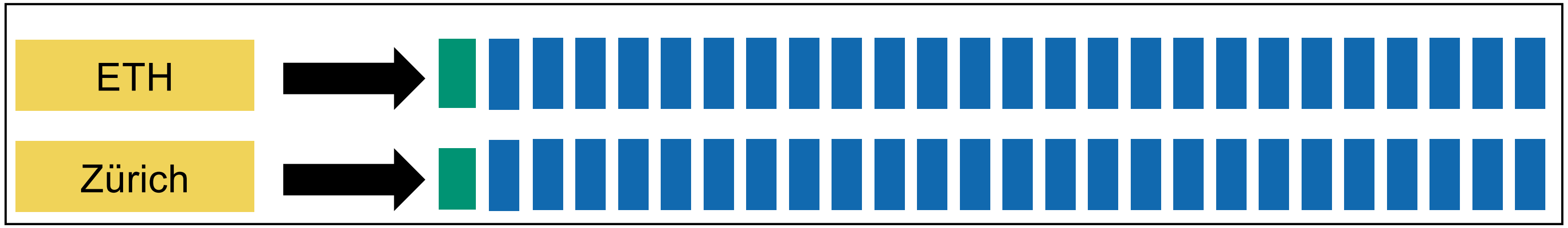
Additional benefit: low-idf terms have more documents

Index elimination: first idea

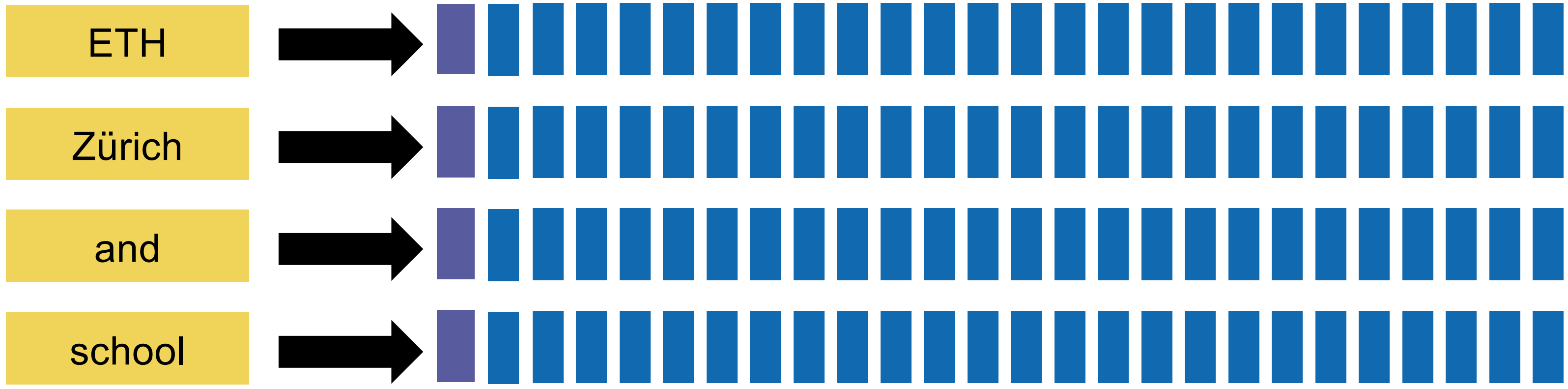


Stop words!

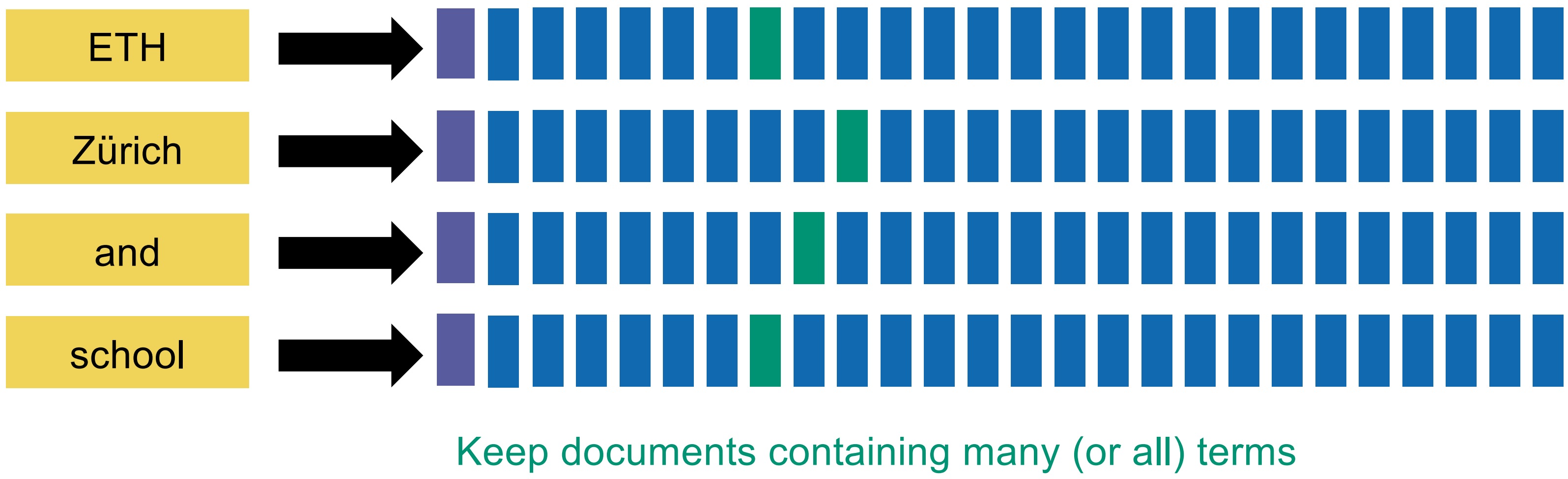
Index elimination: first idea



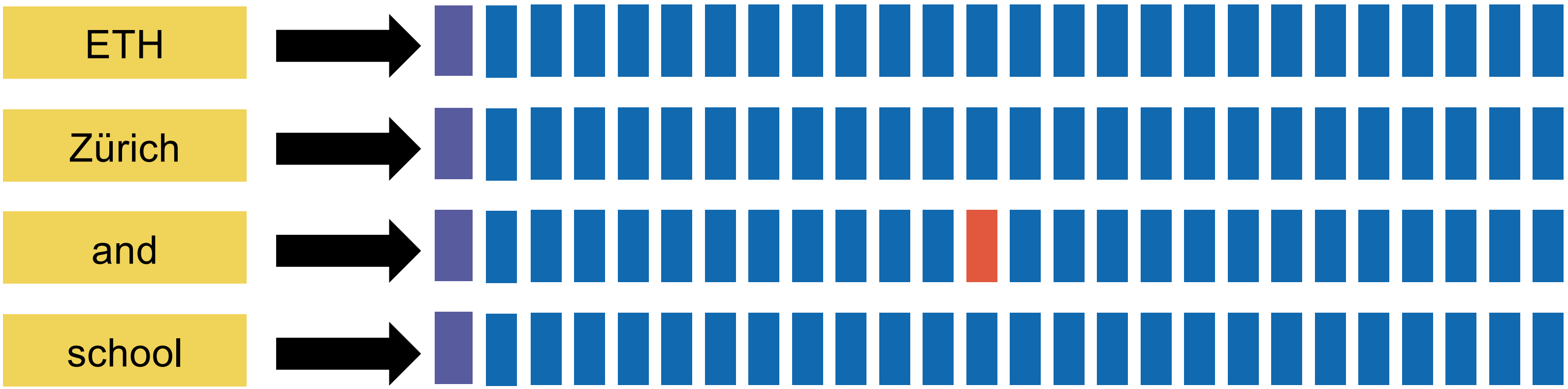
Index elimination: second idea



Index elimination: second idea

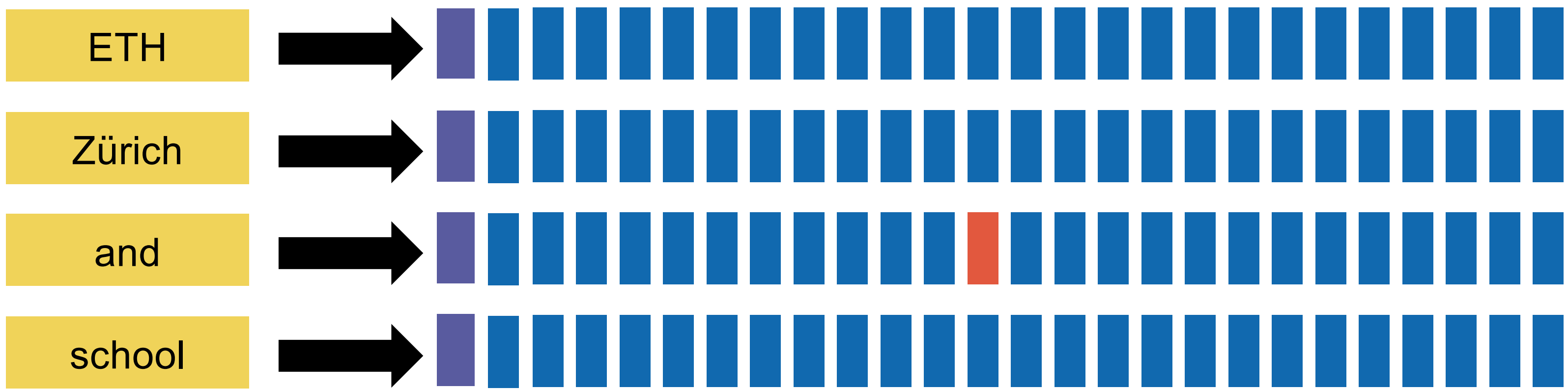


Index elimination: second idea



Drop documents containing only a few terms

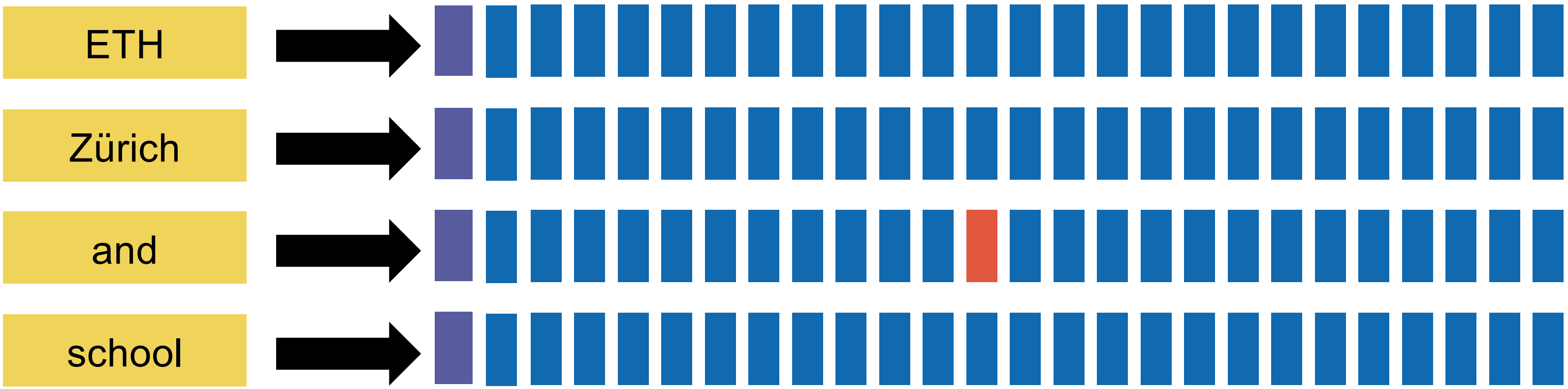
Index elimination: second idea



Drop documents containing only a few terms

(Easy to figure out using intersection algorithm)

Index elimination: second idea

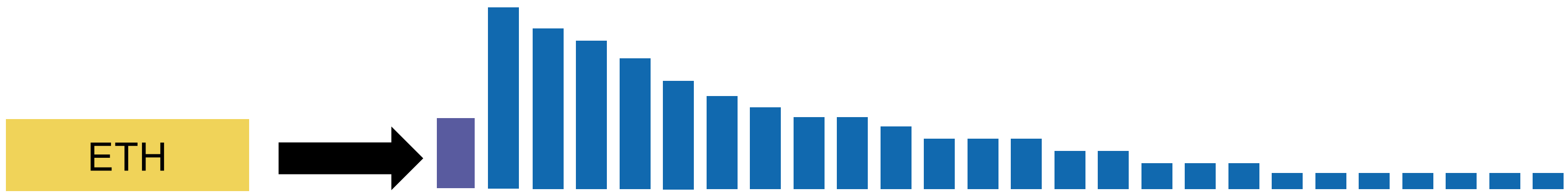


Drop documents containing only a few terms

Issue: we may eliminate **too many** documents.

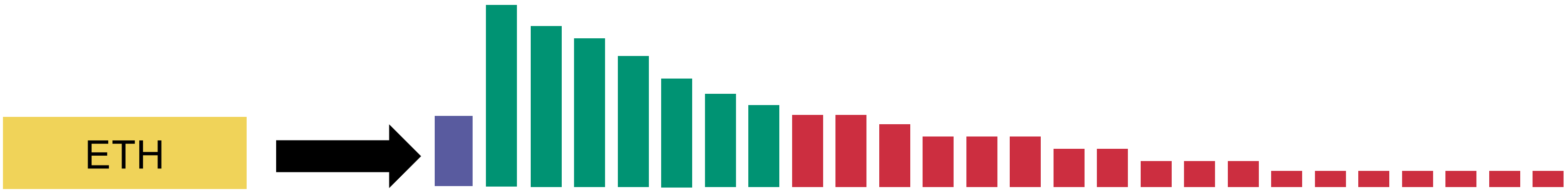
Champion lists

Champion lists



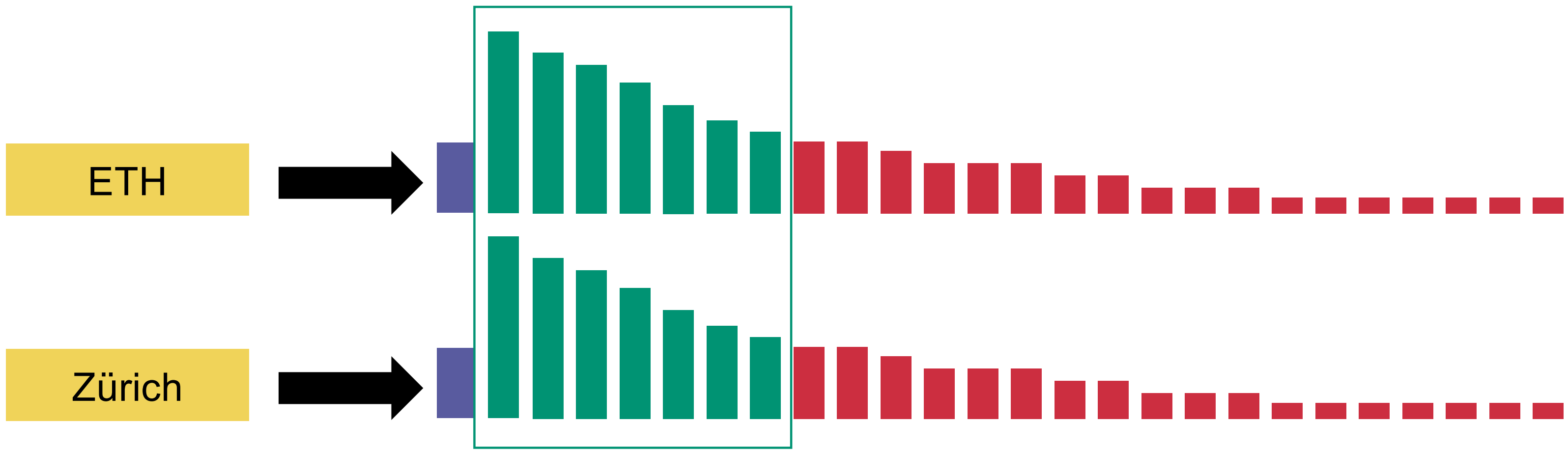
Sort by decreasing term frequency

Champion lists



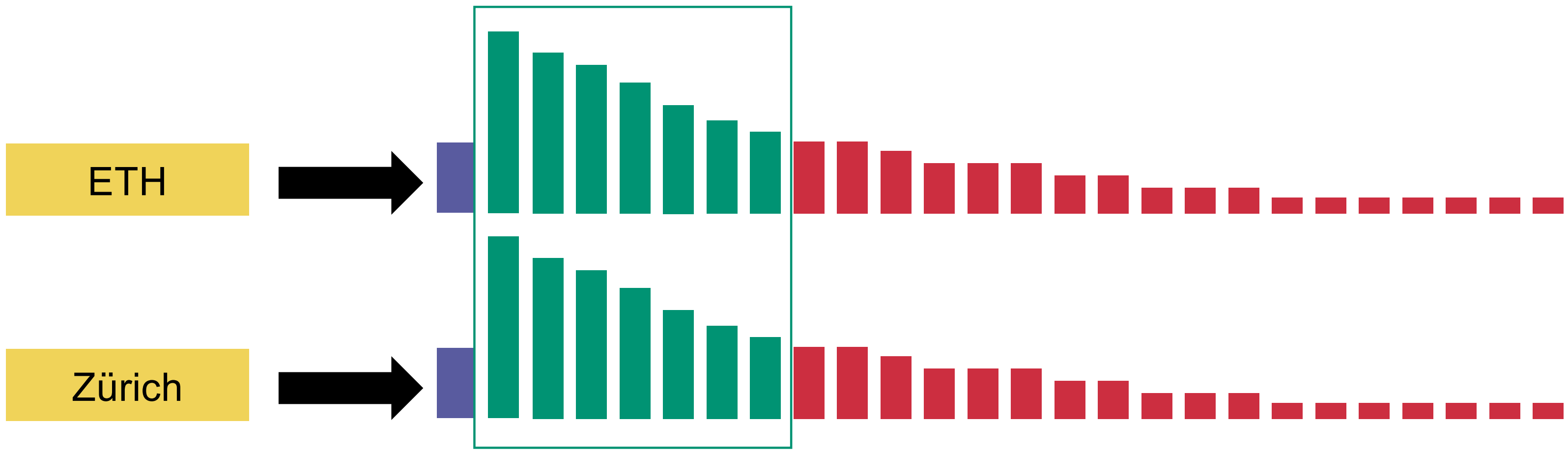
Only keep the top r documents

Champion lists



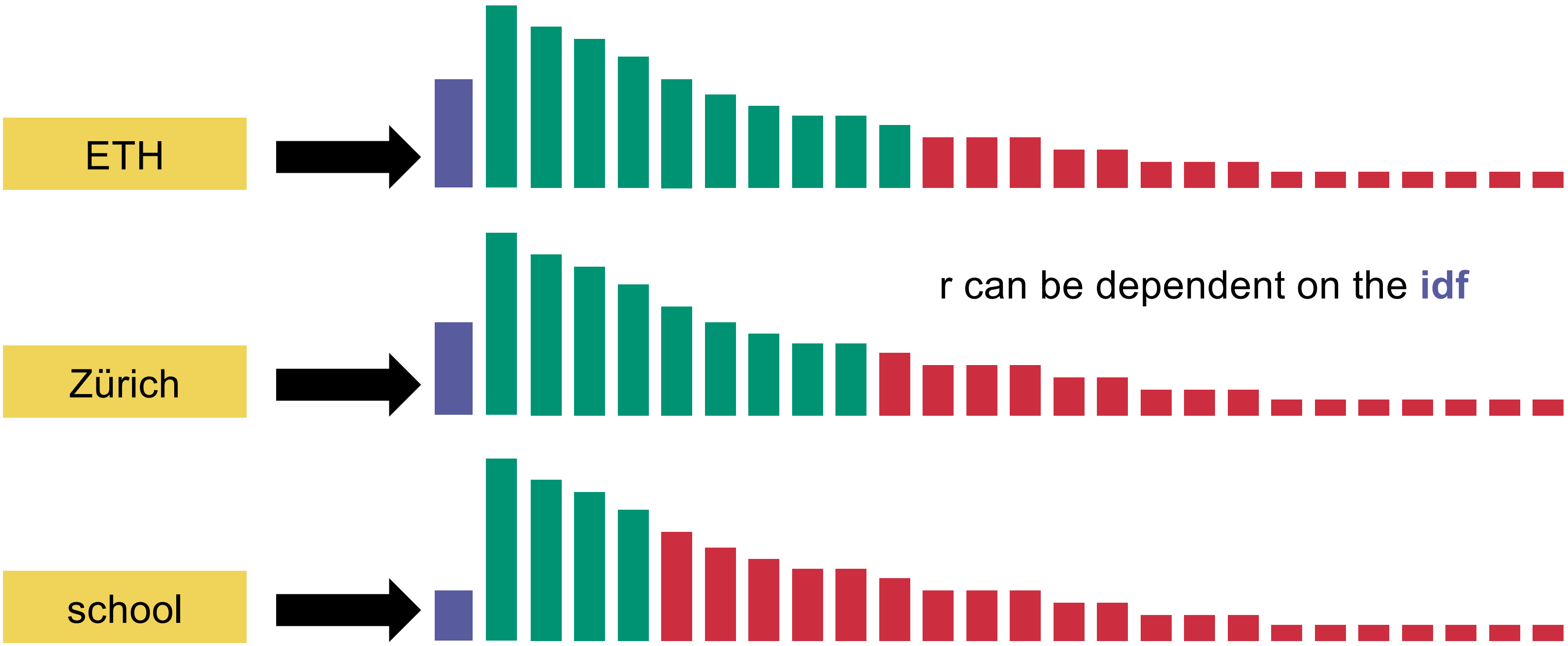
Union the top r from each term

Champion lists

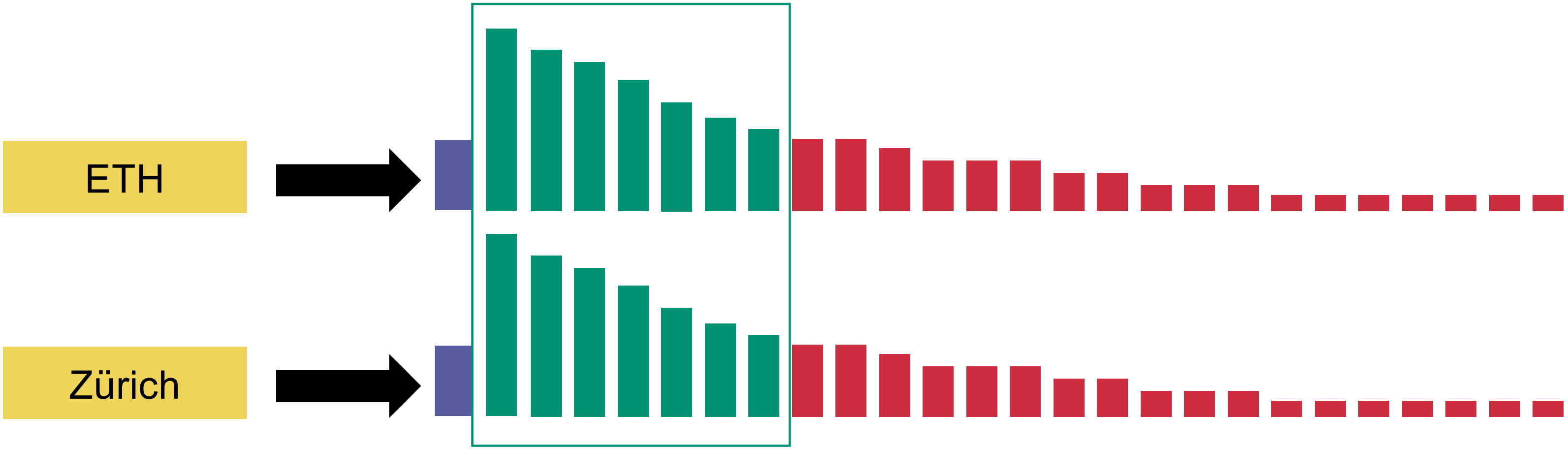


r is chosen **statically**

Champion lists



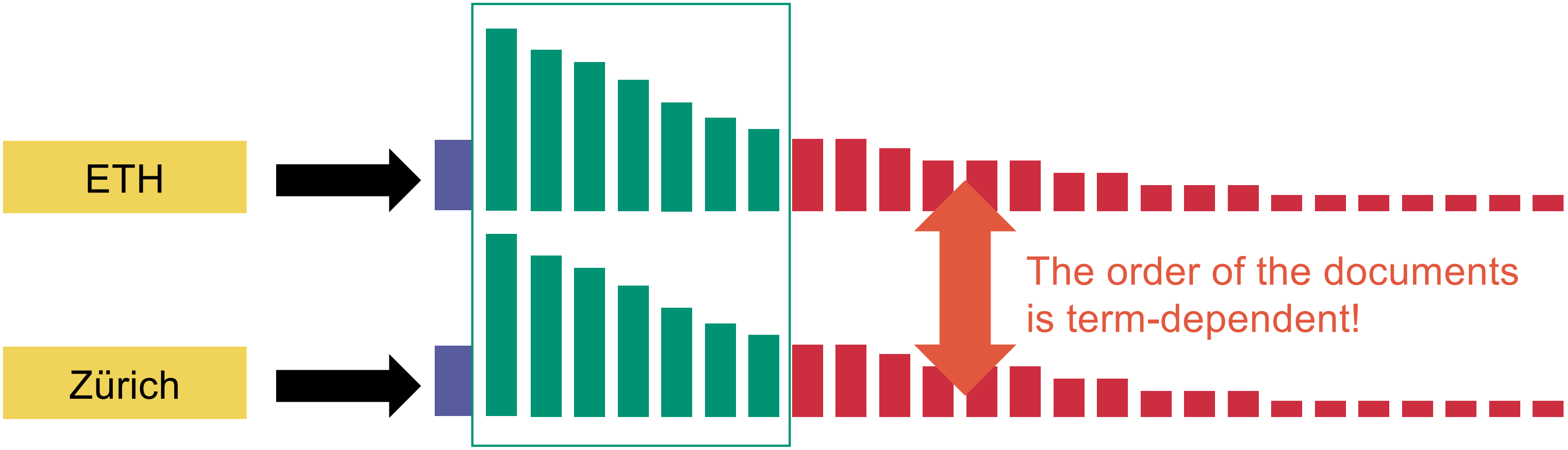
Champion lists



Union the top r from each term

What is the difficulty here?

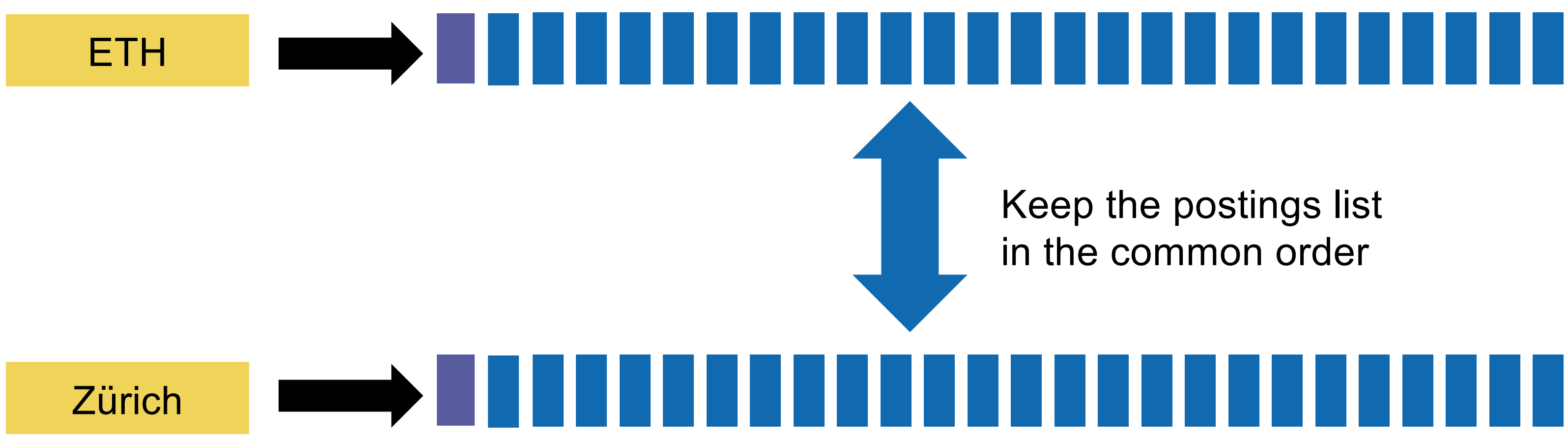
Champion lists



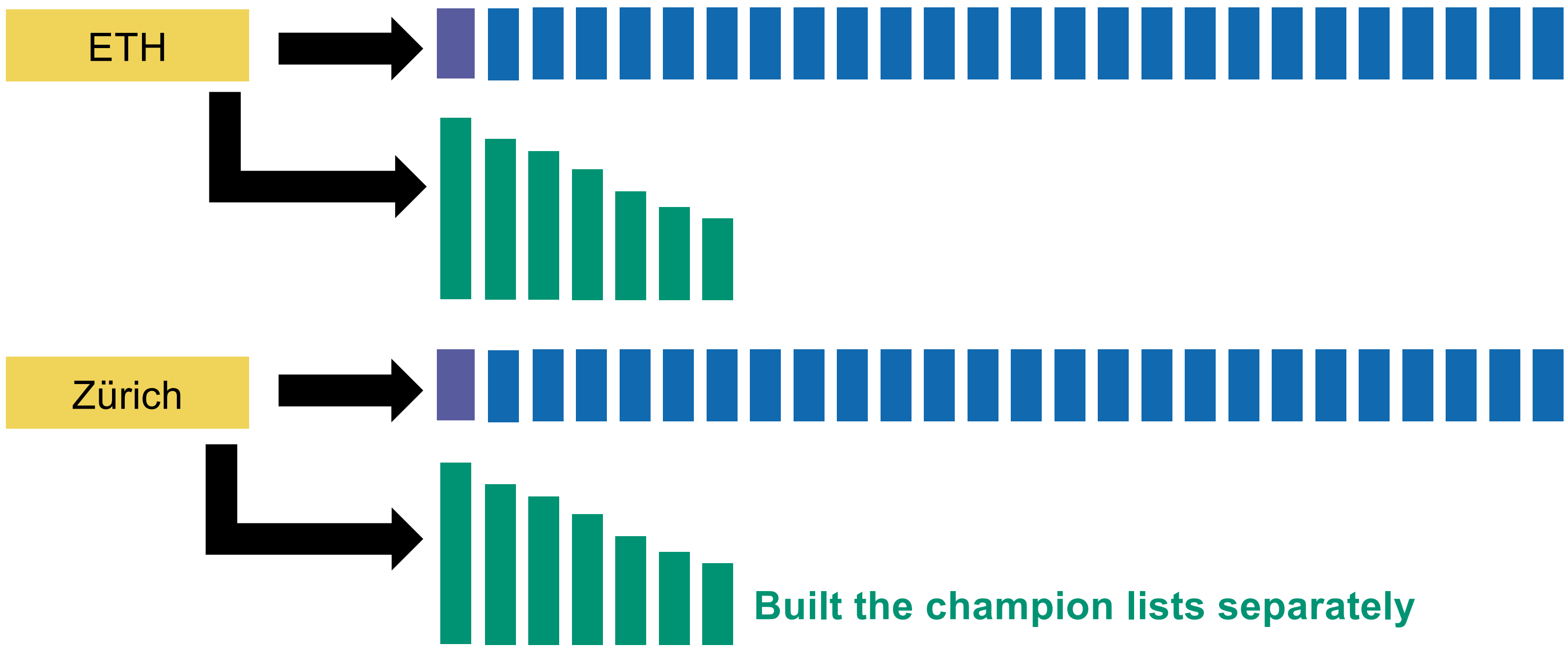
Intersect the top r from each term

What is the difficulty here?

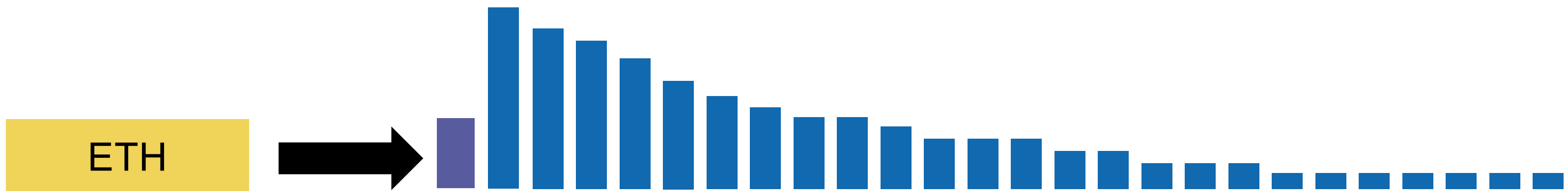
Champion lists



Champion lists

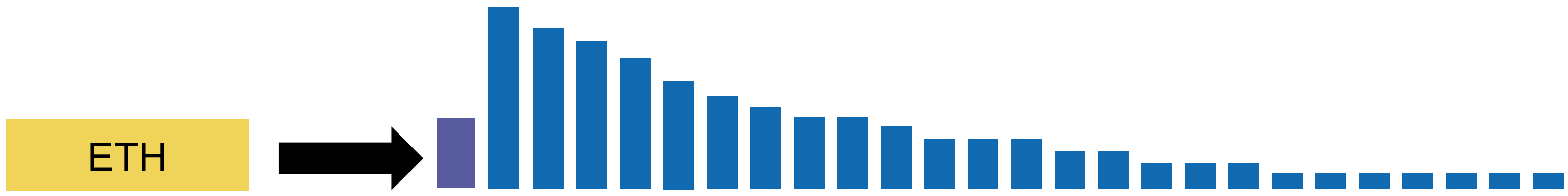


Static Quality Scores



We assign each document d a **quality score** g(d) in [0,1]

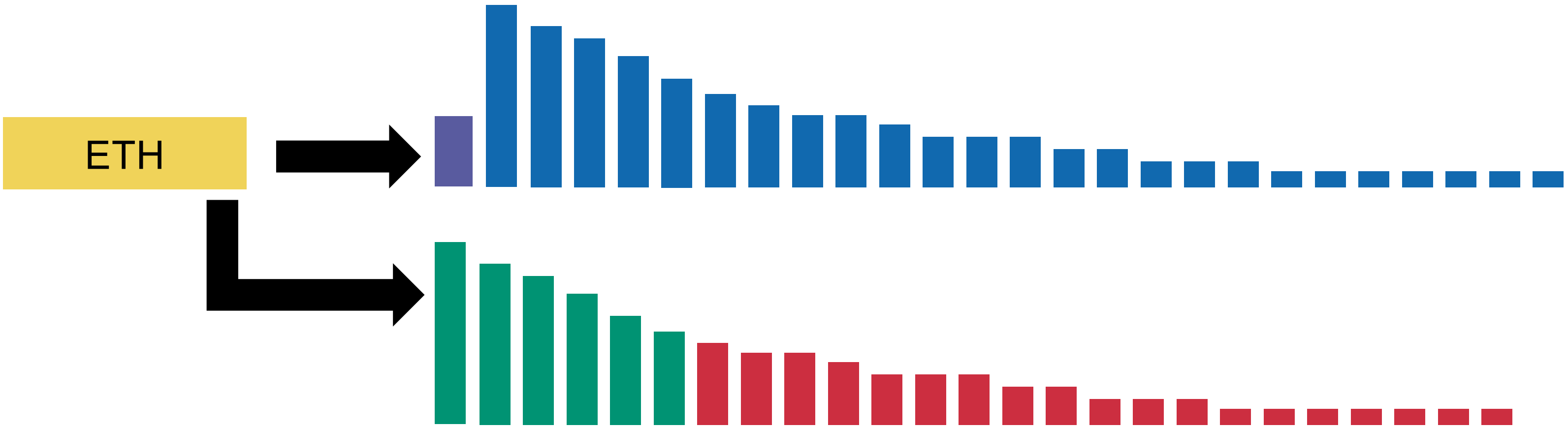
Static Quality Scores



We assign each document d a **quality score** g(d) in [0,1]

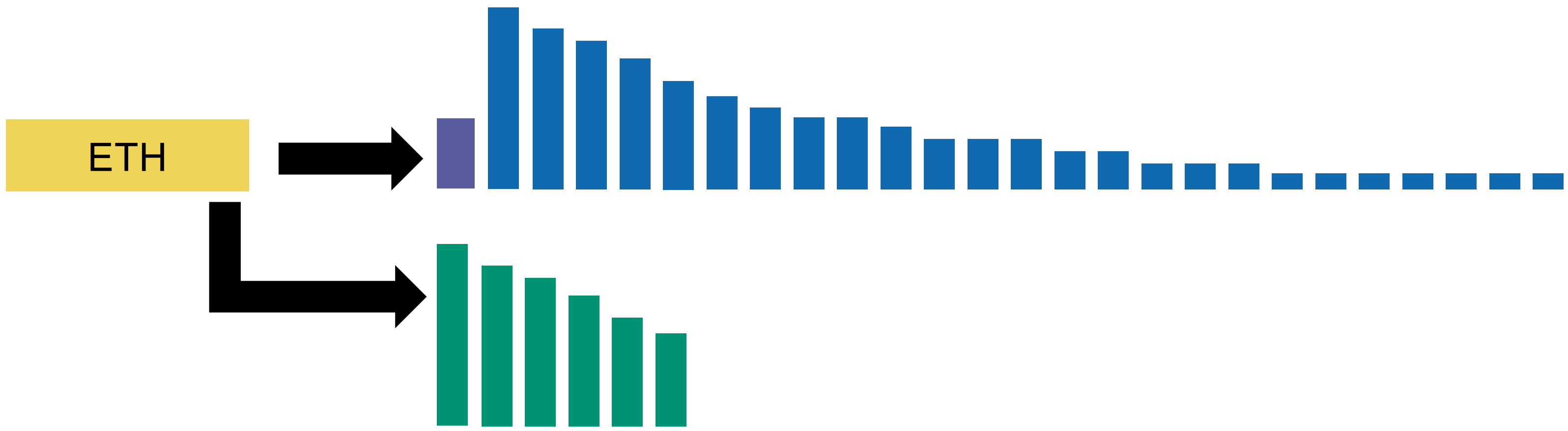
We sort **statically** by g (d)

Static Quality Scores



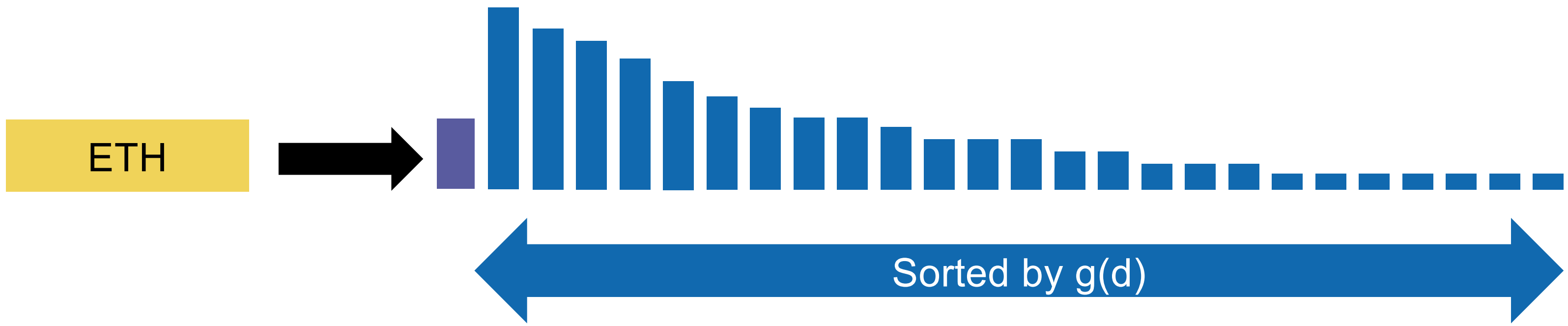
We can build our champion lists with the top g(d) + tf-idf$_{t,d}$

Static Quality Scores

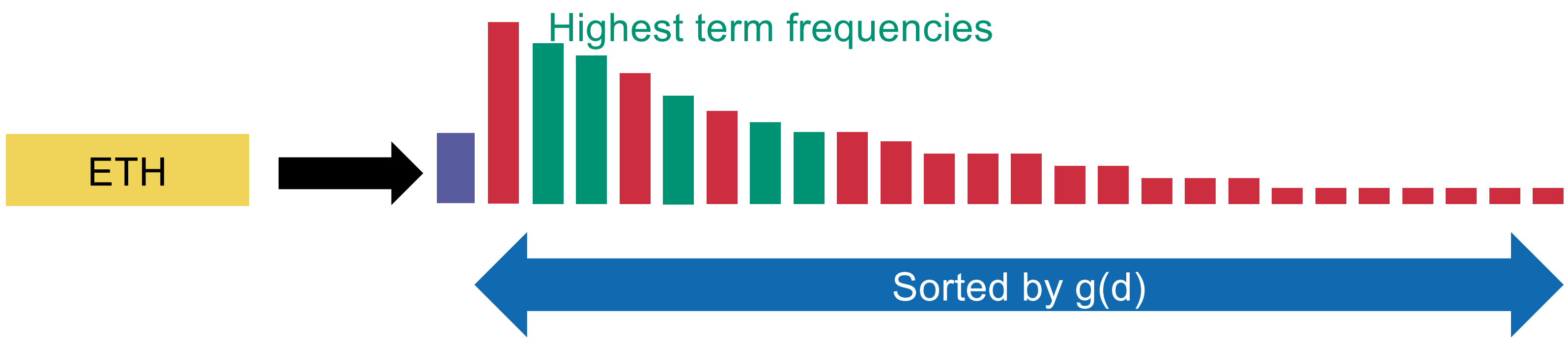


We can build our champion lists with the top g(d) + tf-idf$_{t,d}$

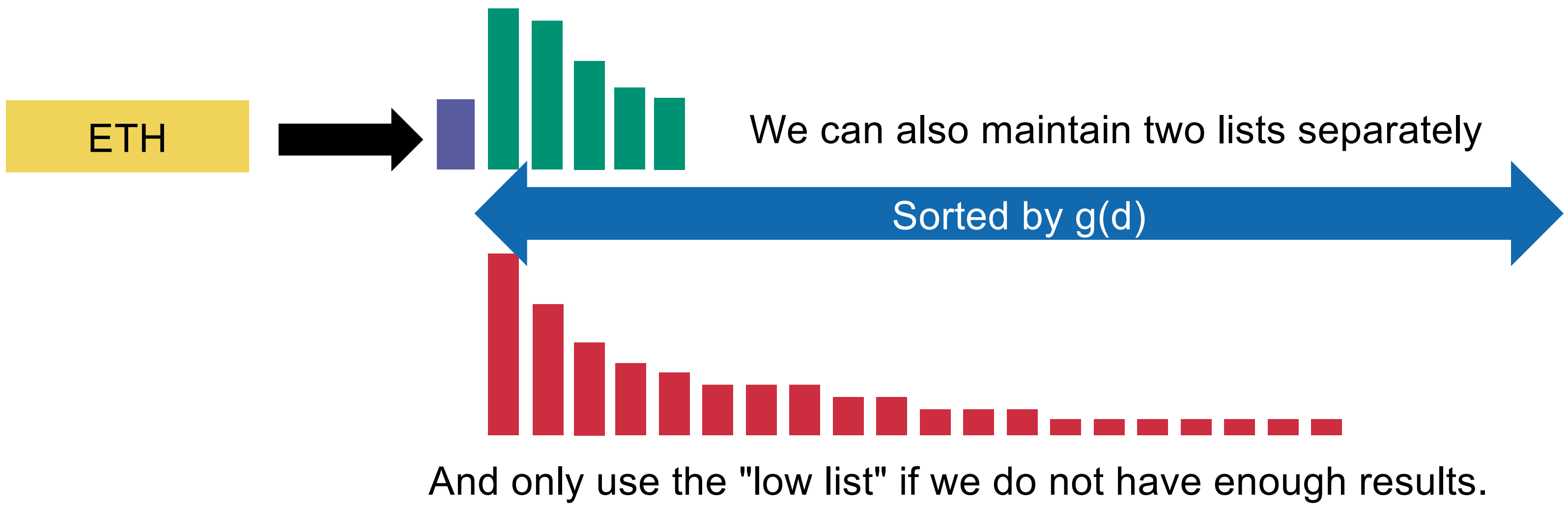
Static Quality Scores



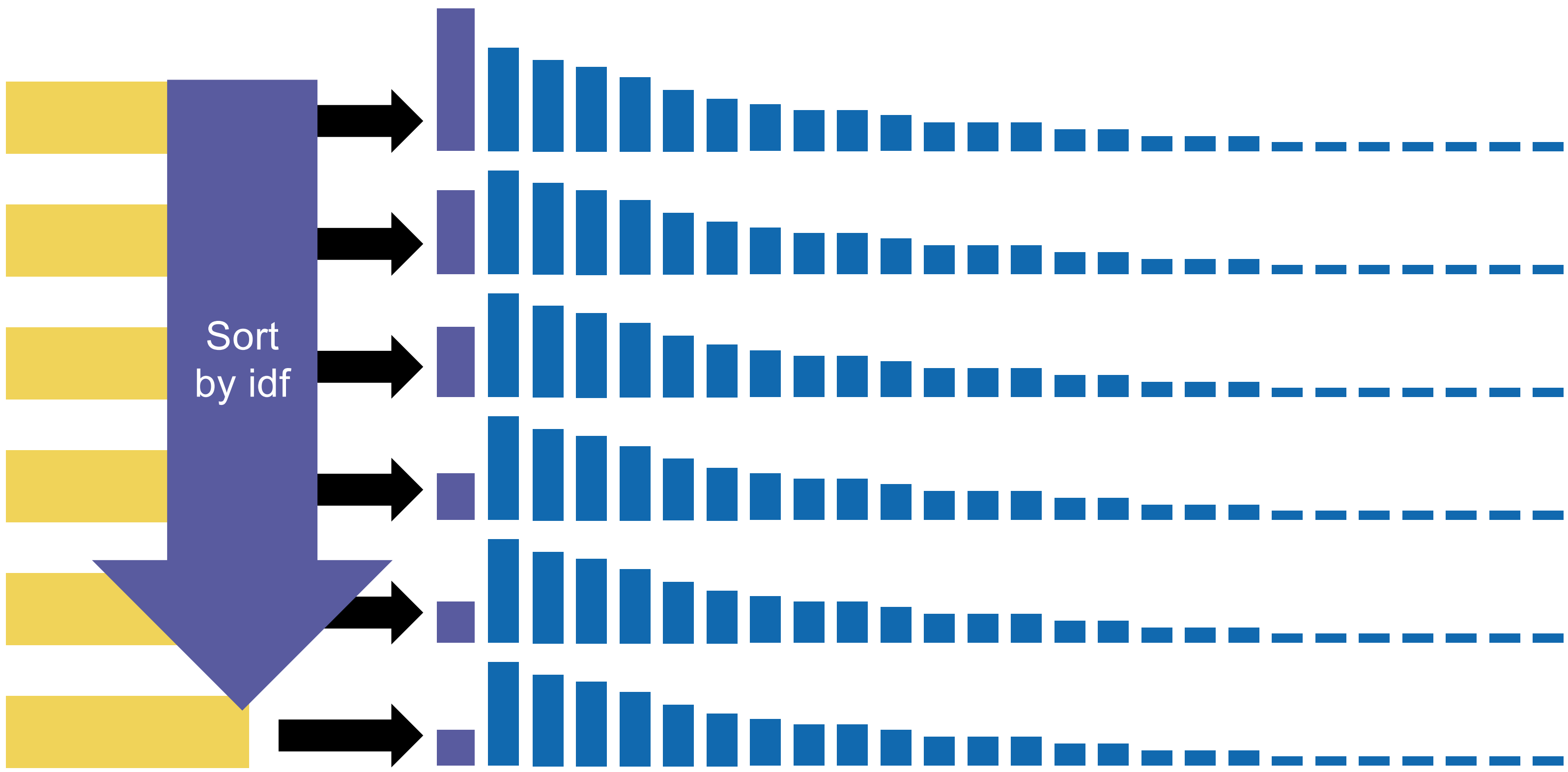
Static Quality Scores



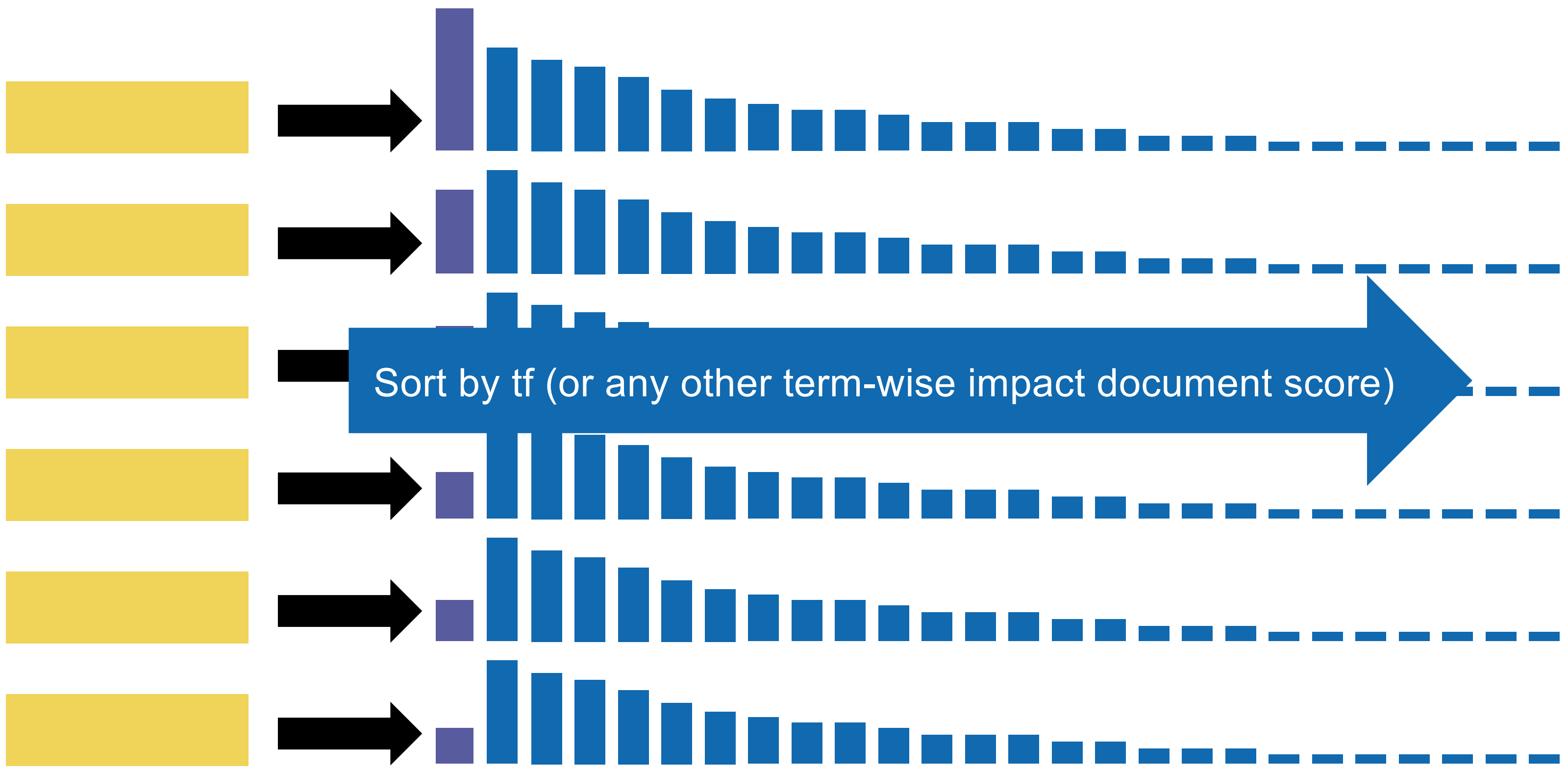
Static Quality Scores



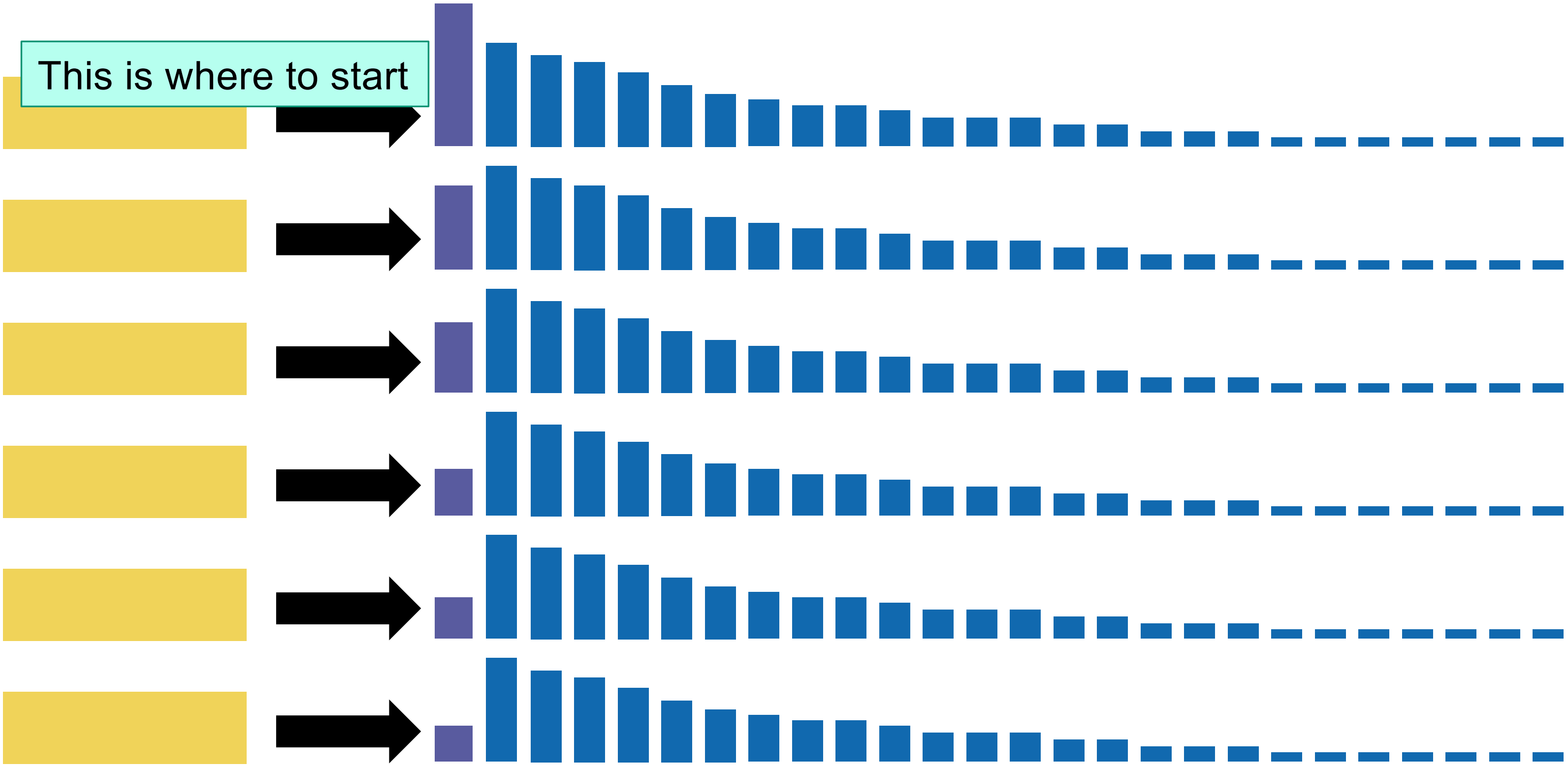
Impact ordering



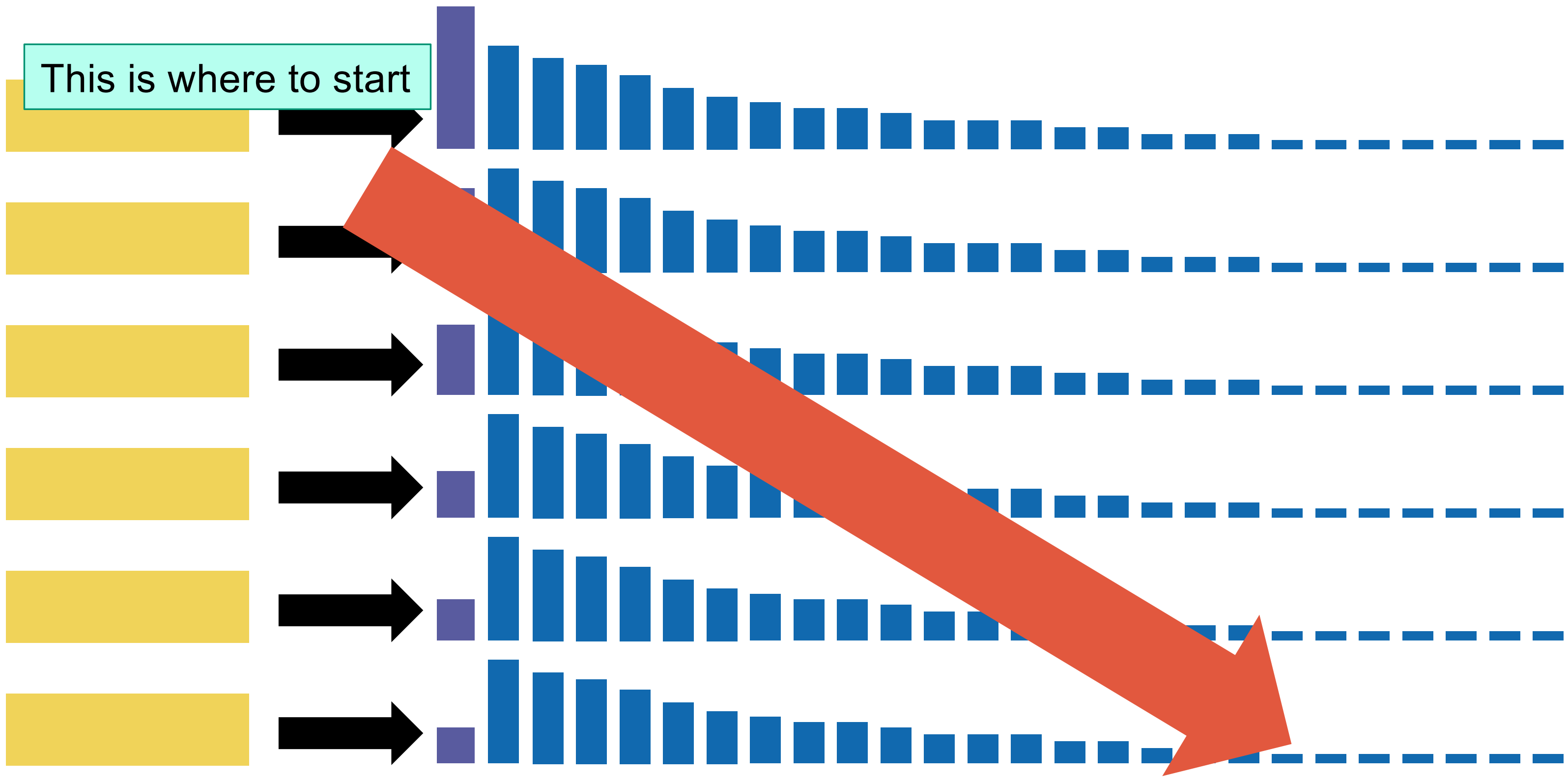
Impact ordering



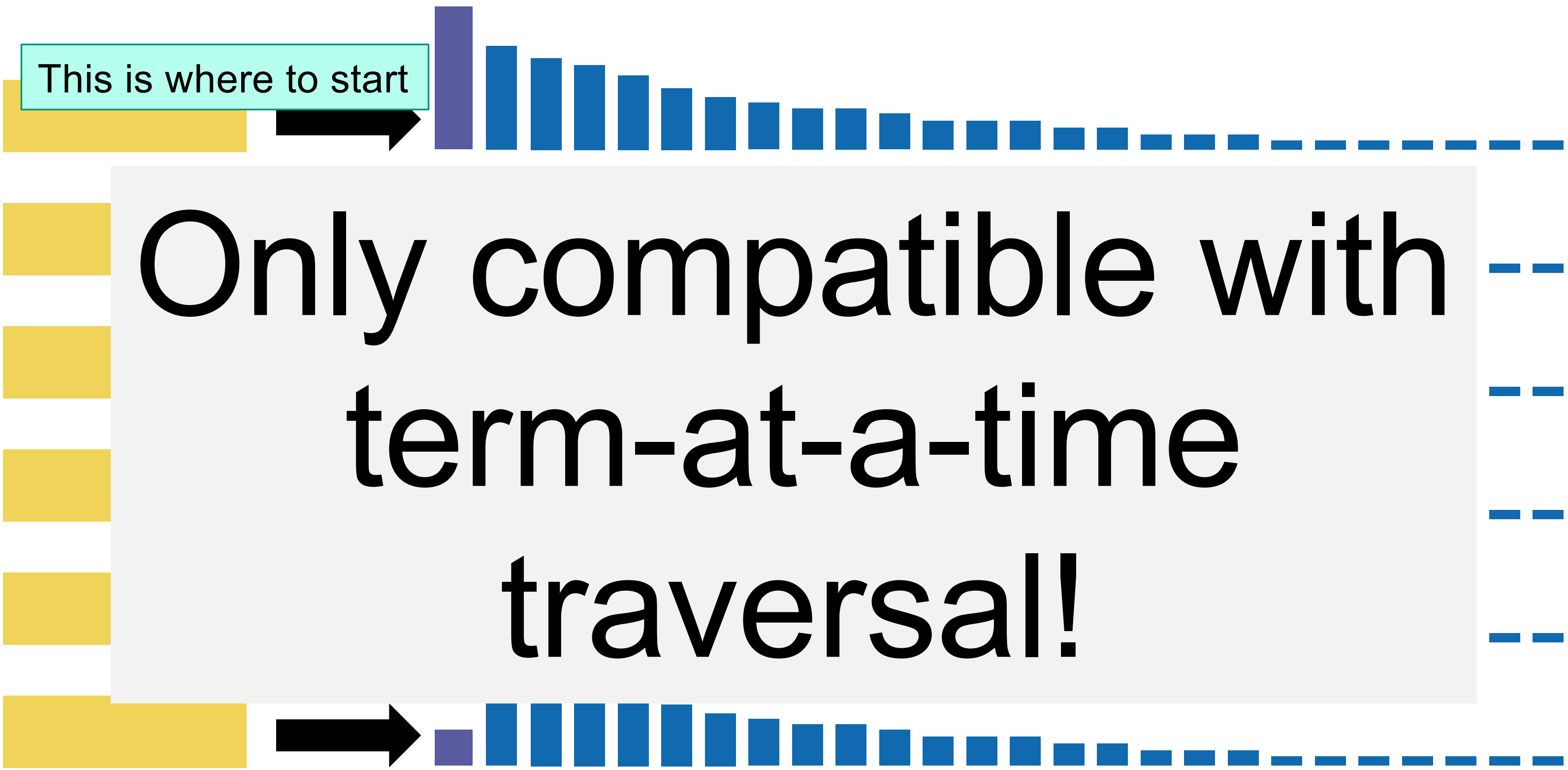
Impact ordering



Impact ordering

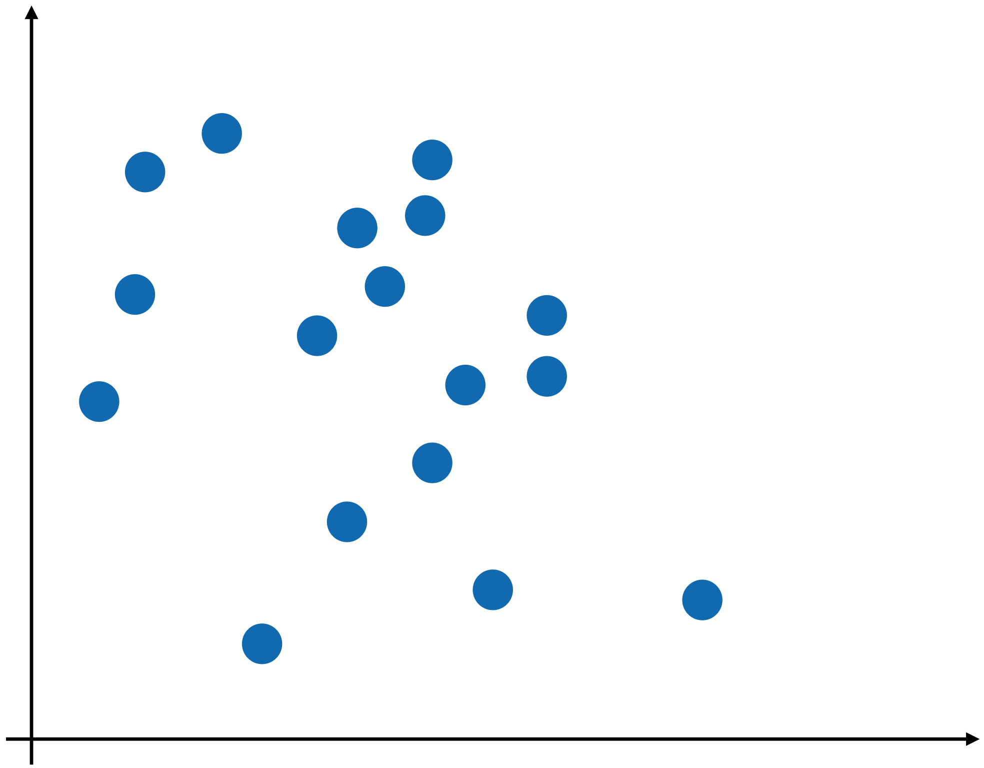


Impact ordering

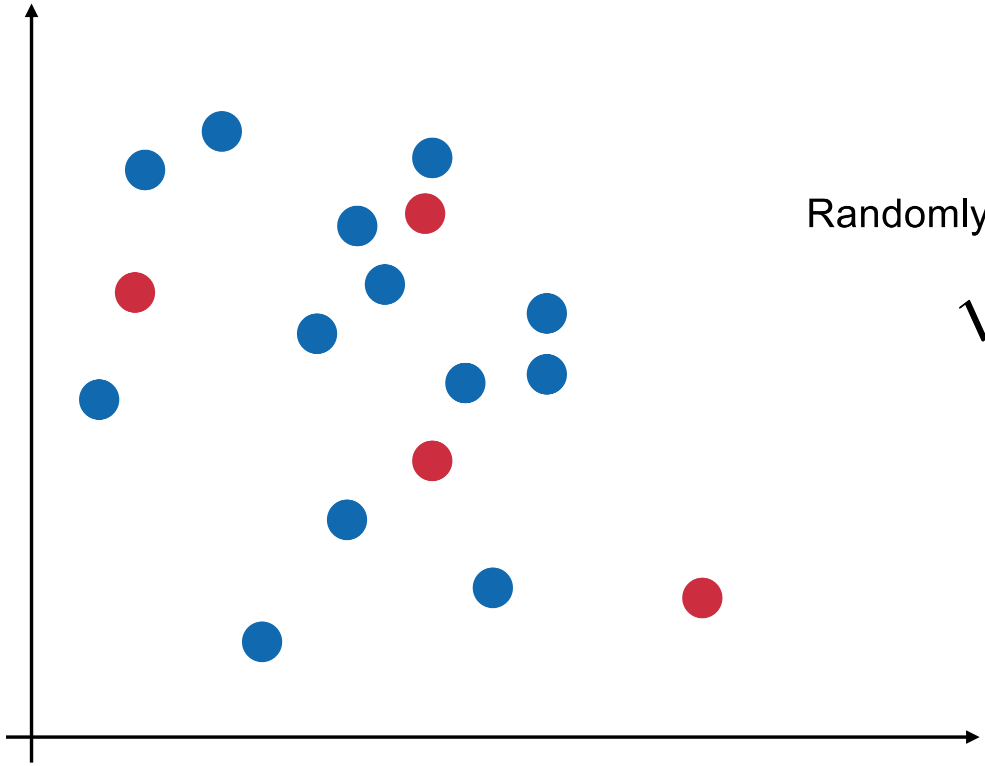


Only compatible with term-at-a-time traversal!

Clustering



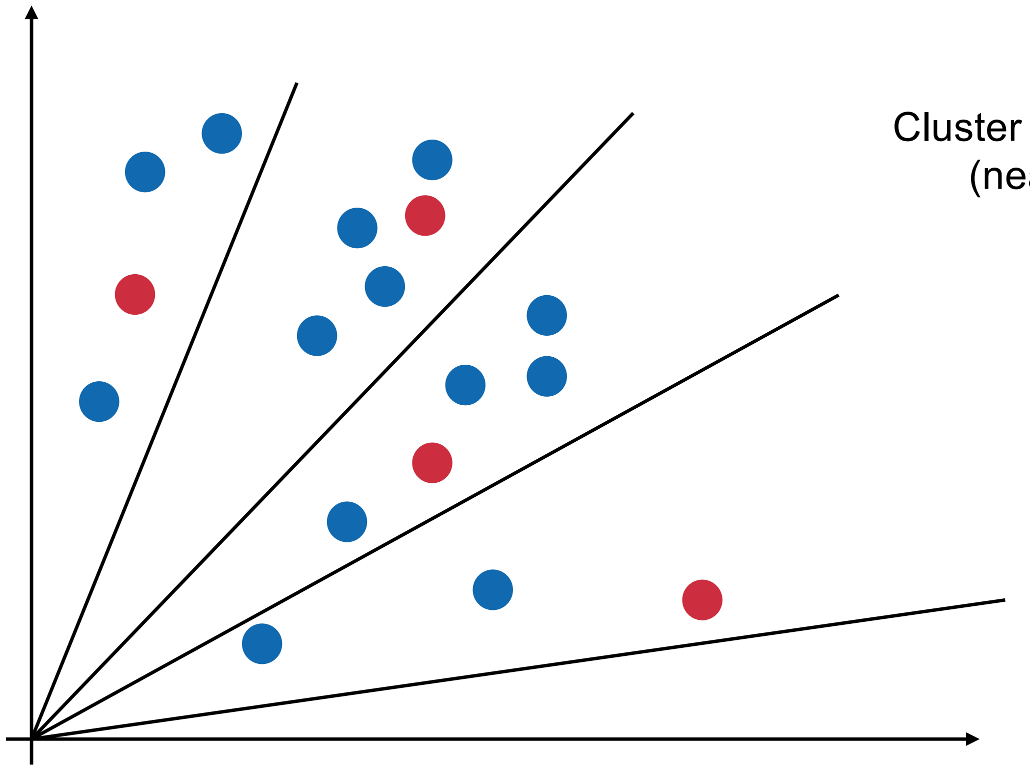
Clustering



Randomly pick leaders

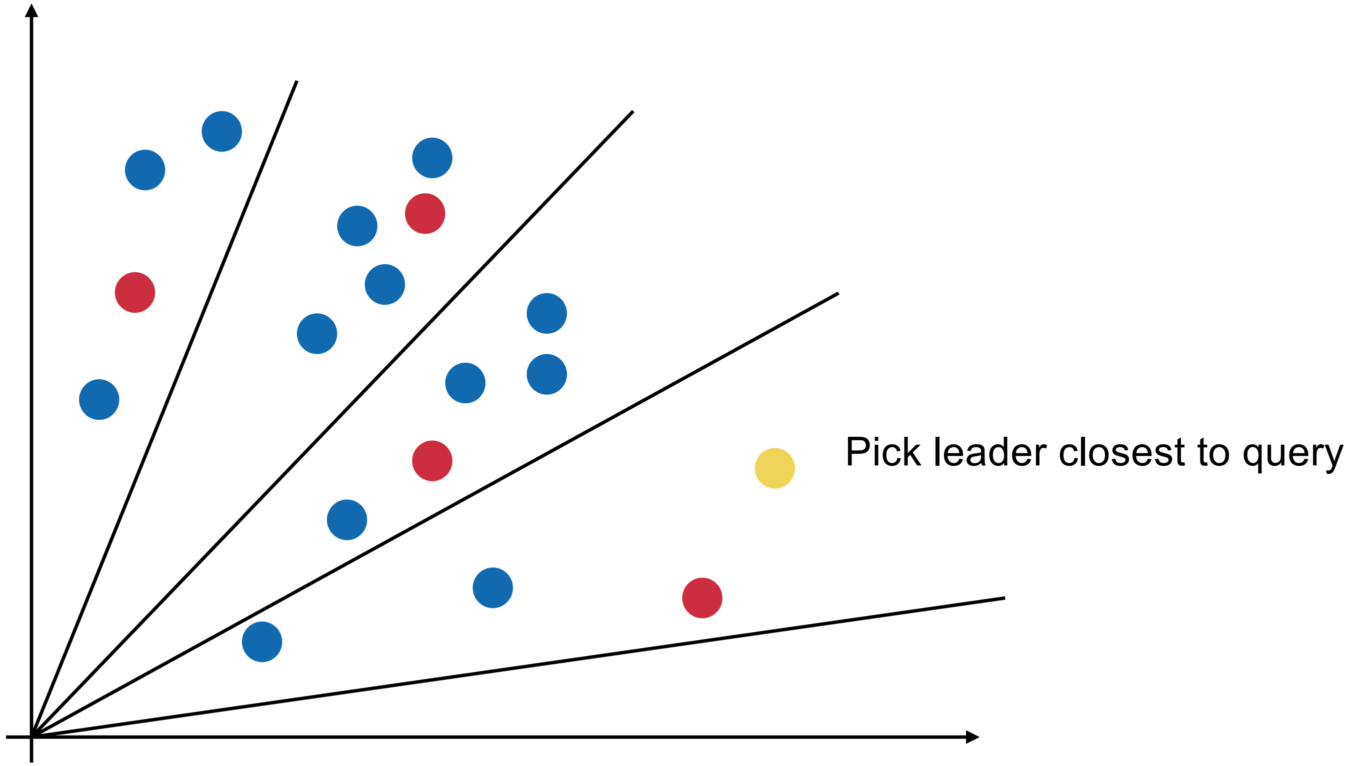
$$\sqrt{N}$$

Clustering

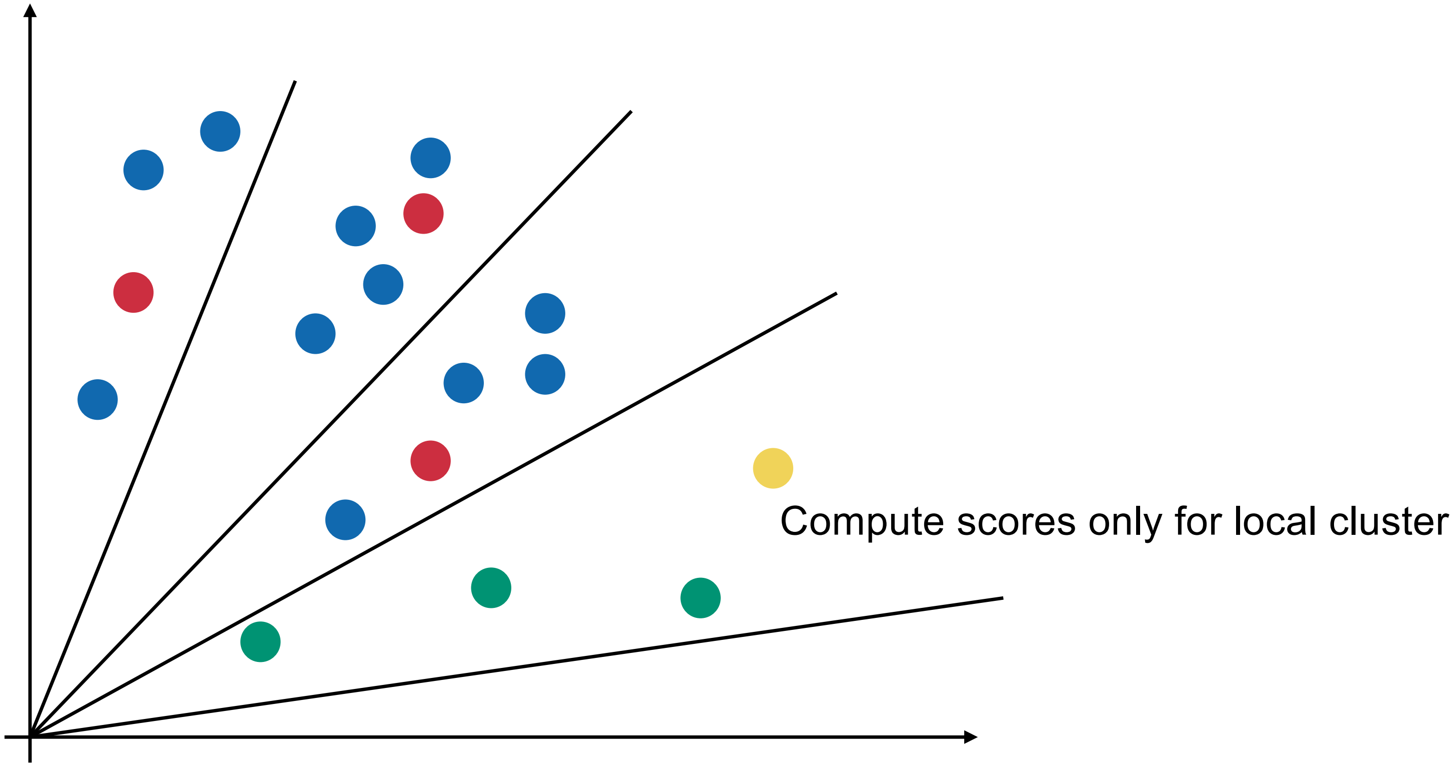


Cluster document space
(nearest leader)

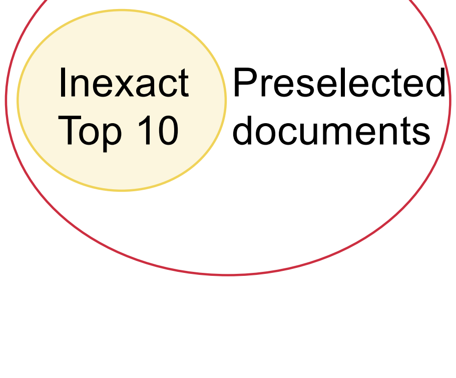
Clustering



Clustering

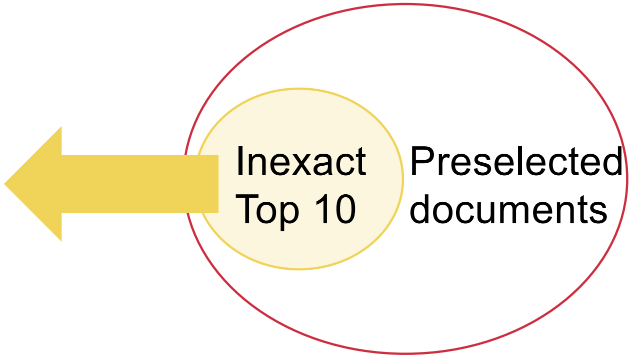


Tiered indices

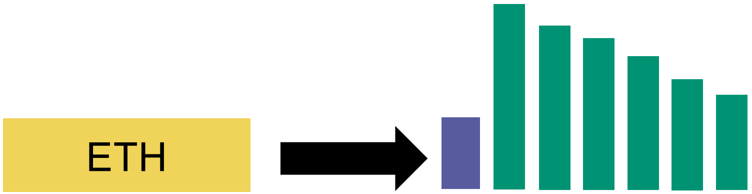


Tiered indices

What if
this gets us
less
than 10
documents
?



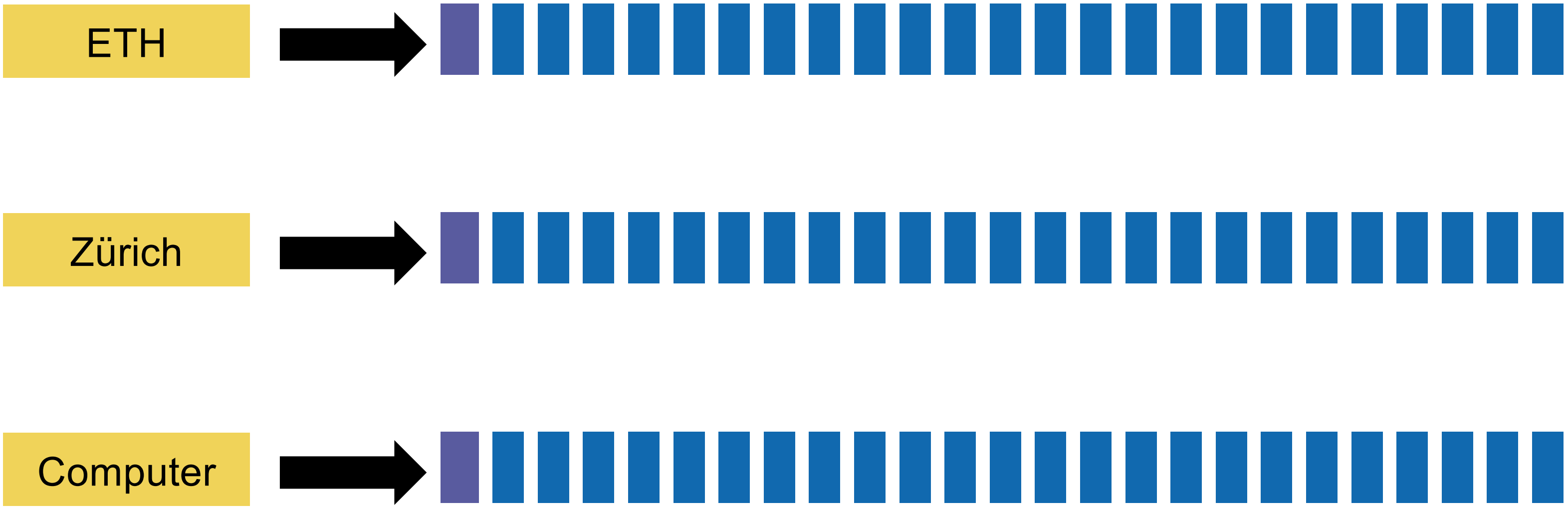
Reminder: Quality Scores



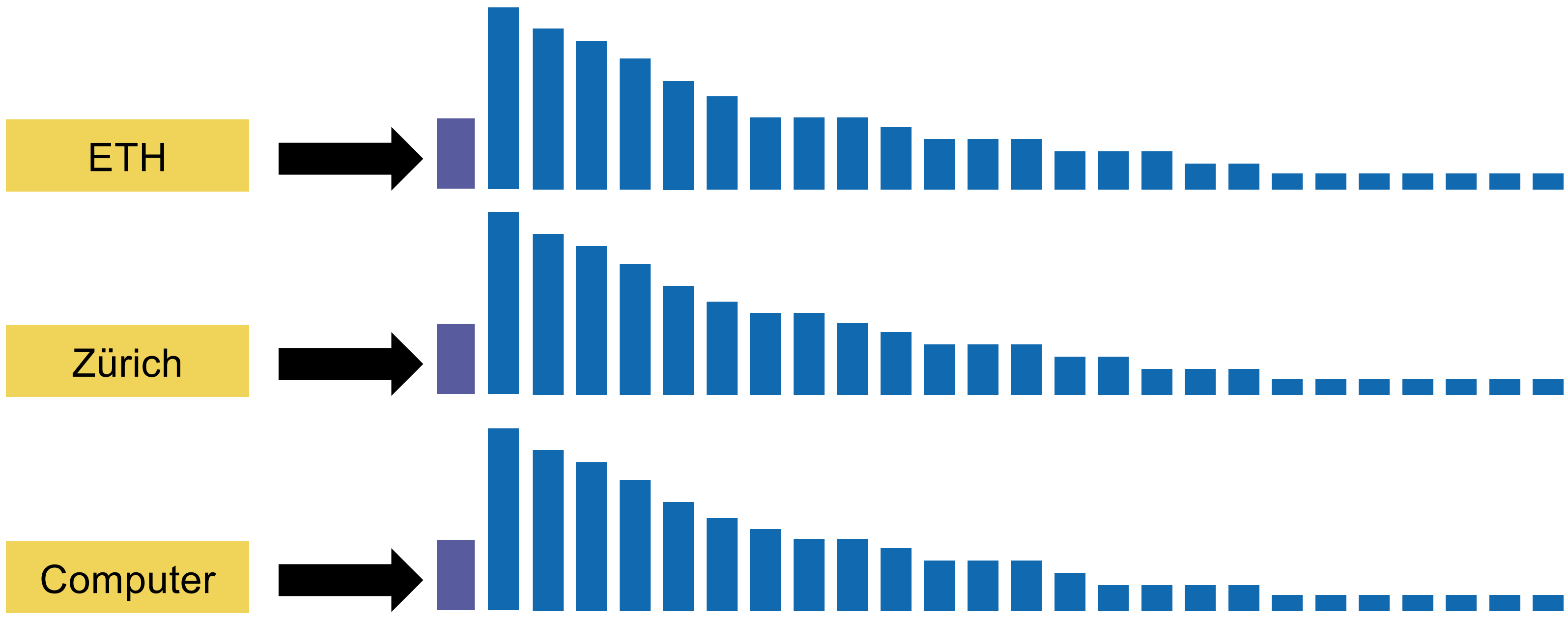
We can also maintain two lists separately

And only use the "low list" if we do not have enough results.

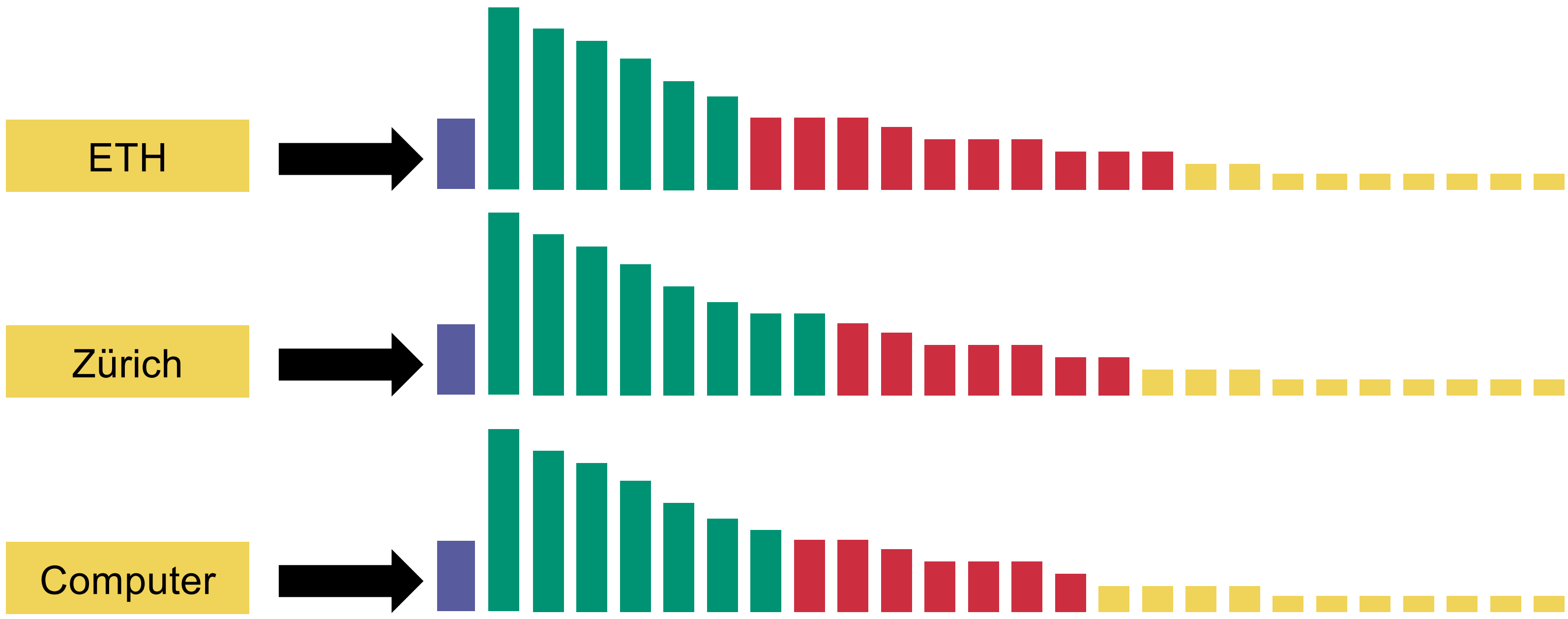
Tiered indices



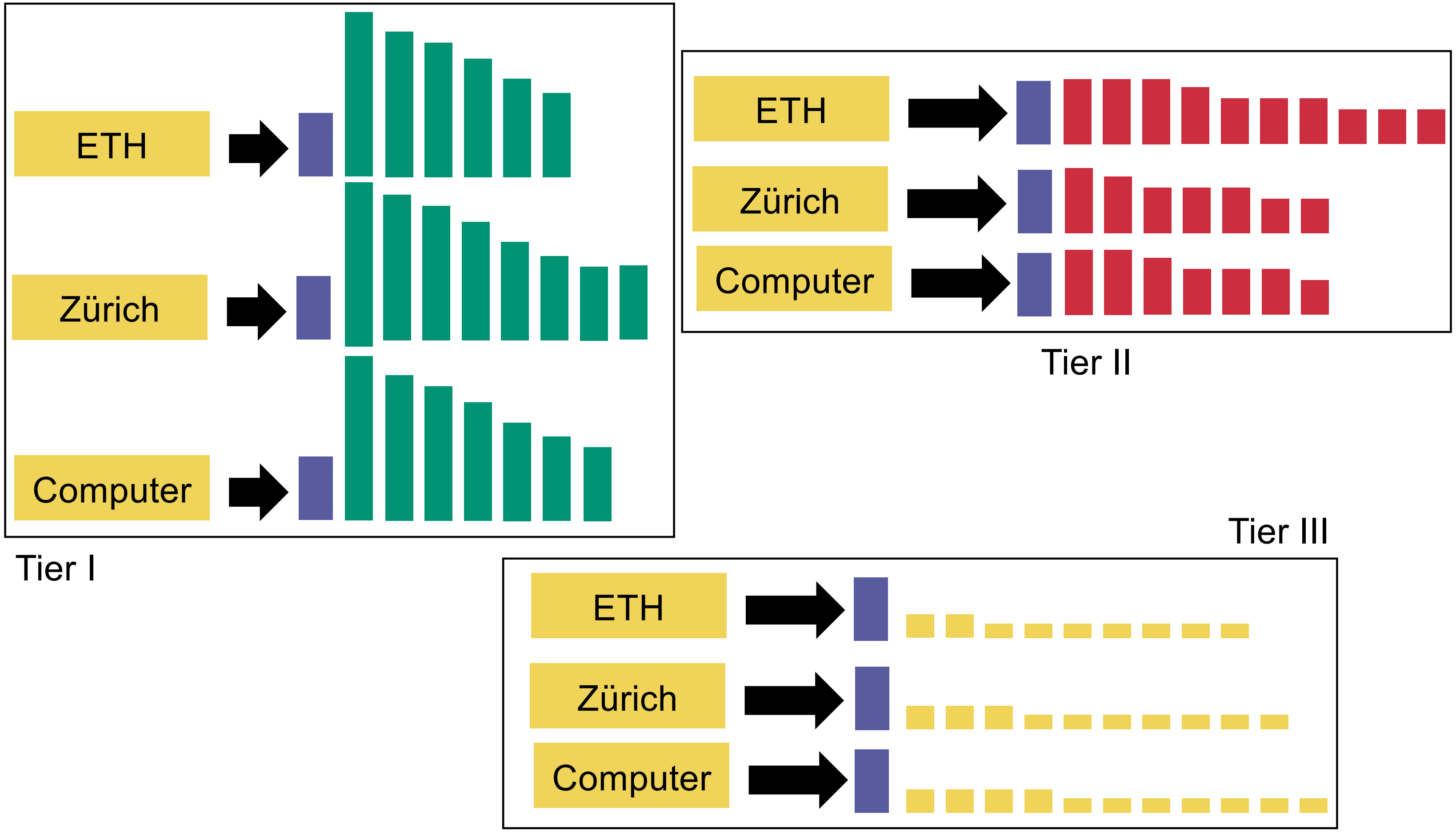
Tiered indices



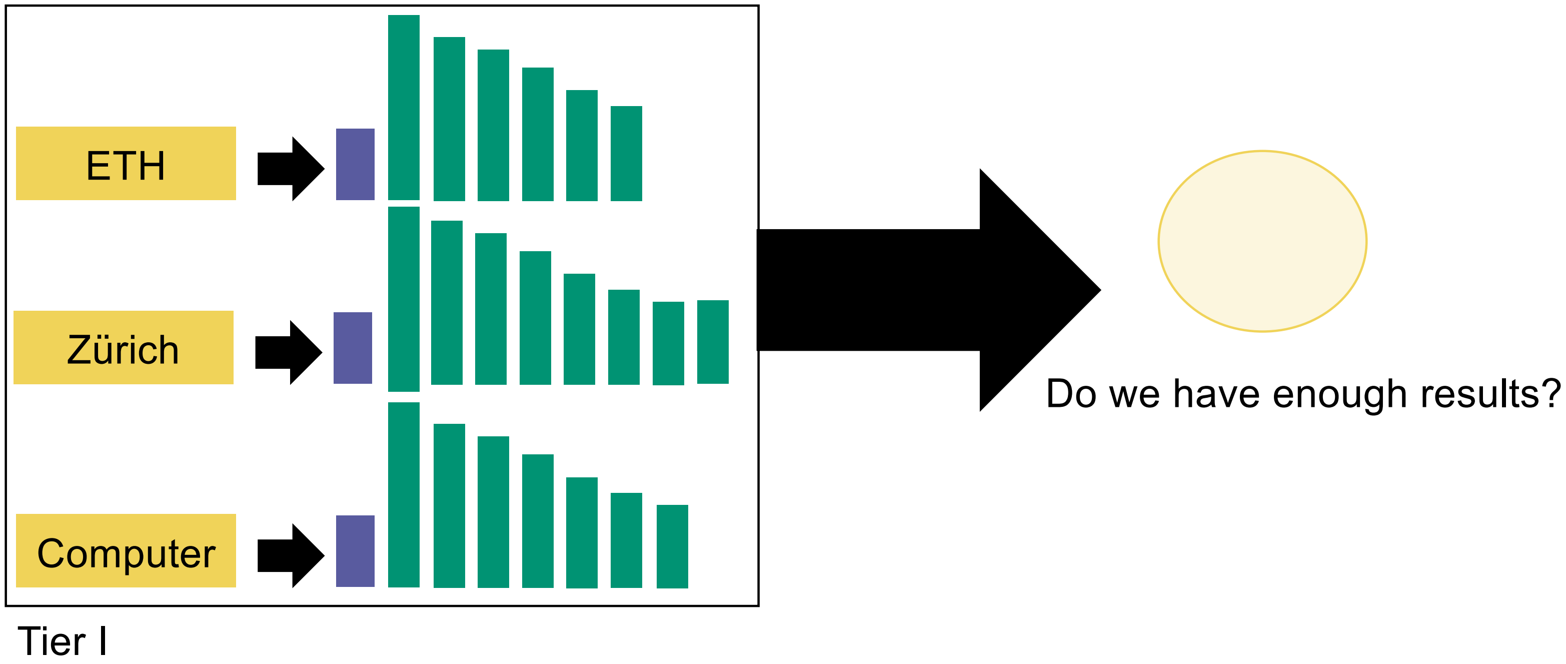
Tiered indices



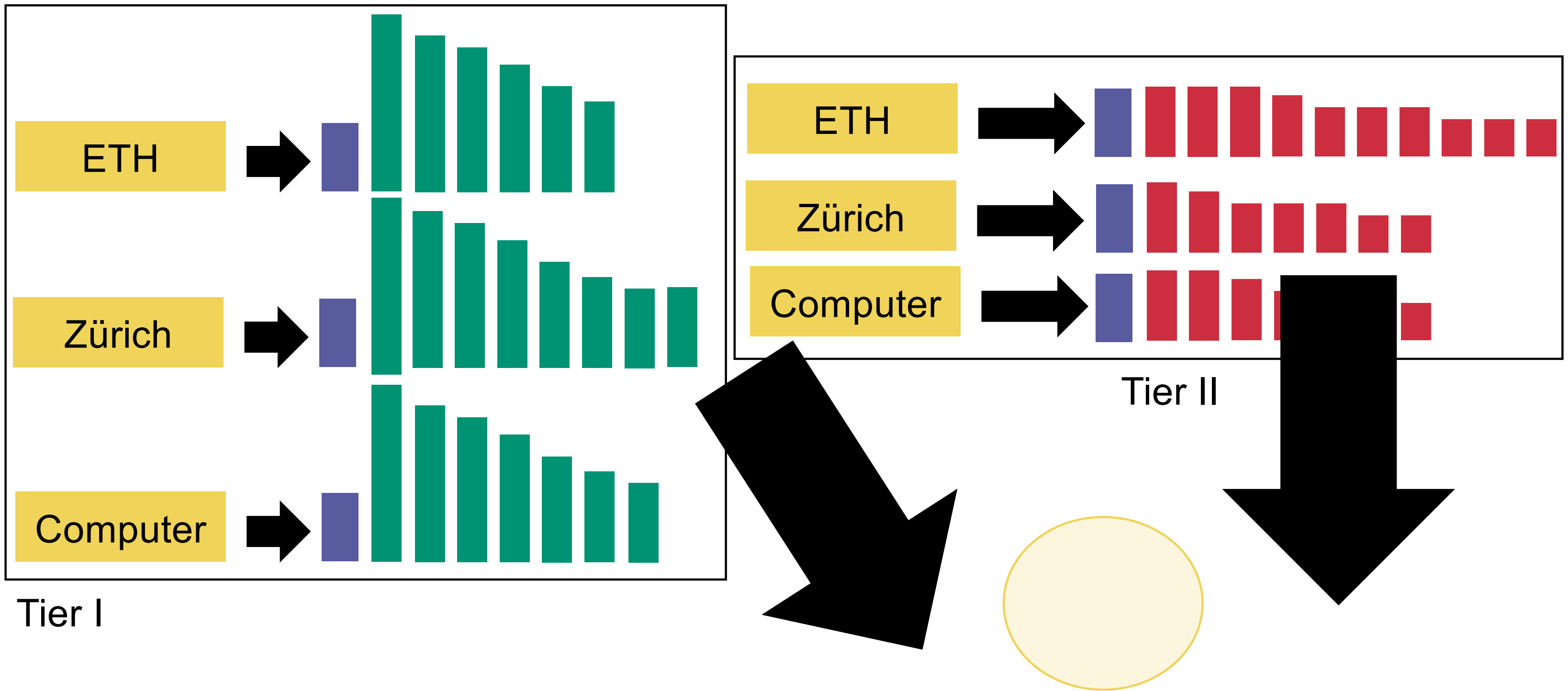
Tiered indices



Tiered indices

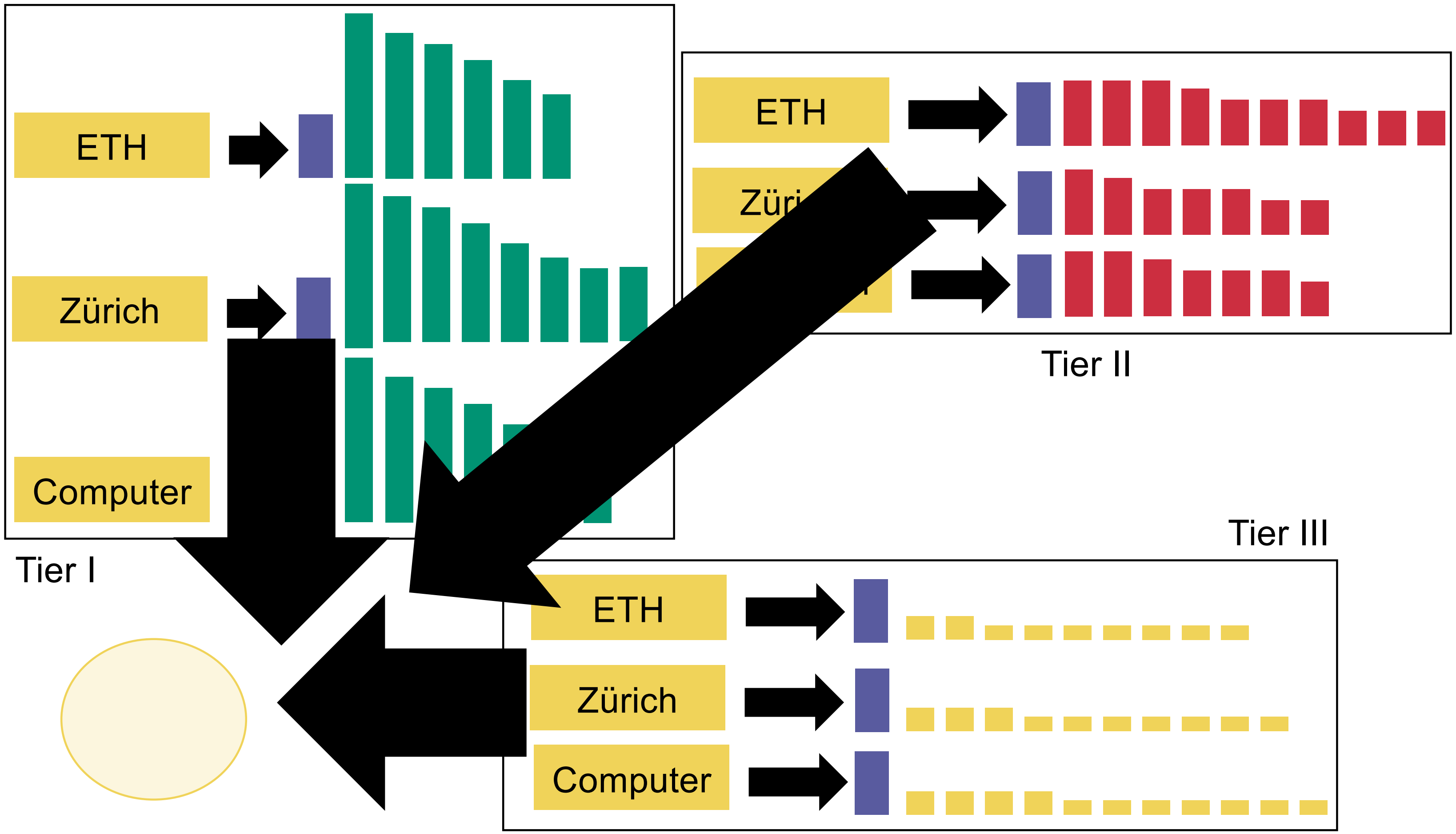


Tiered indices



Do we have enough results?

Tiered indices



Query term proximity

ETH Zurich

Query term proximity

$\boxed{\texttt{ETH Zurich}}$

One of the two Swiss ETH is in Zurich.

Query term proximity

`ETH Zurich`

One of the two Swiss ETH is in Zurich.

ω=4

Query term proximity

`ETH Zurich`

There is an active ETH cryptocurrency ecosystem in Zug, near Zurich.

Query term proximity

`ETH Zurich`

There is an active ETH cryptocurrency ecosystem in Zug, near Zurich.

ω=7

Query term proximity

`ETH Zurich`

ETH Zurich has a department of Computer Science.

Query term proximity

`ETH Zurich`

$\omega=2$

ETH Zurich has a department of Computer Science.

Query term proximity

`ETH Zurich`

4

One of the two Swiss ETH is in Zurich.

7

There is an active ETH cryptocurrency ecosystem in Zug, near Zurich.

2

ETH Zurich has a department of Computer Science.

Query term proximity

`ETH Zurich`

2 ETH Zurich has a department of Computer Science.

4 One of the two Swiss ETH is in Zurich.

7 There is an active ETH cryptocurrency ecosystem in Zug, near Zurich.

Query term proximity

`ETH Zurich`

ETH Zurich has a department of Computer Science.

One of the two Swiss ETH is in Zurich.

Top 2

Query interfaces

```
ETH AND Zurich
OR (Computer
AND NOT Science)
```

Boolean queries

```
"ETH Zurich
Computer Science"
```

Phrase queries

```
ETH Z*rich
```

Wildcard queries

```
ETH Zurich
Pascal Algorithm
```

Set of words

Title: `Pascal`
Author: `Niklaus Wirth`

Zone queries

Query interfaces

Inverted index

```
ETH AND Zurich
OR (Computer
AND NOT Science)
```

Boolean queries

k-gram index
permuterm index

```
ETH Z*rich
```

Wildcard queries

Biword indices

```
"ETH Zurich
Computer Science"
```

Phrase queries

Vector-space model

```
ETH Zurich
Pascal Algorithm
```

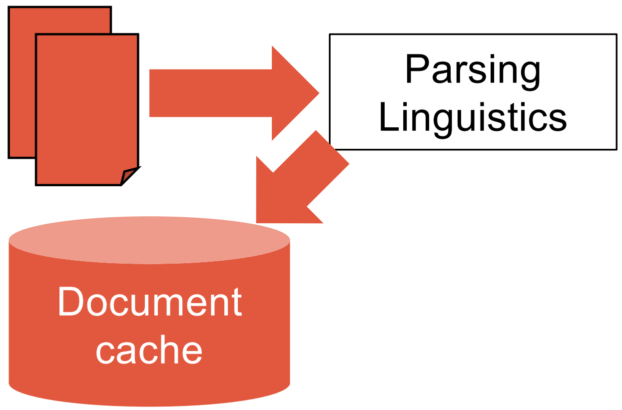
Set of words

Zone indices

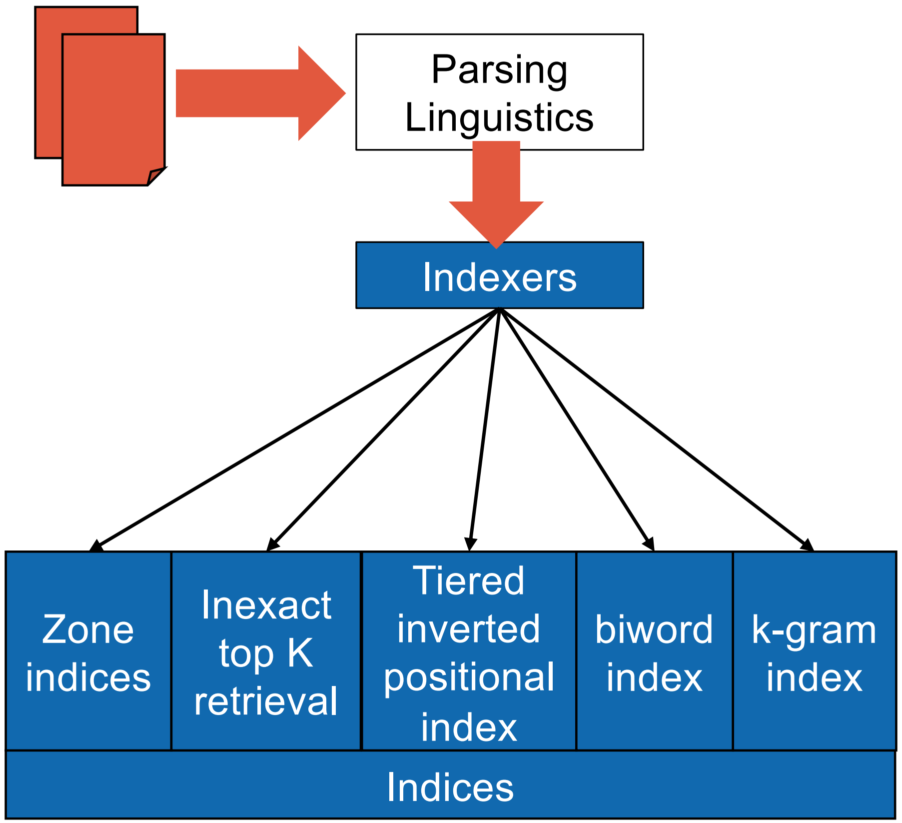
Title: `Pascal`
Author: `Niklaus Wirth`

Field/zone queries

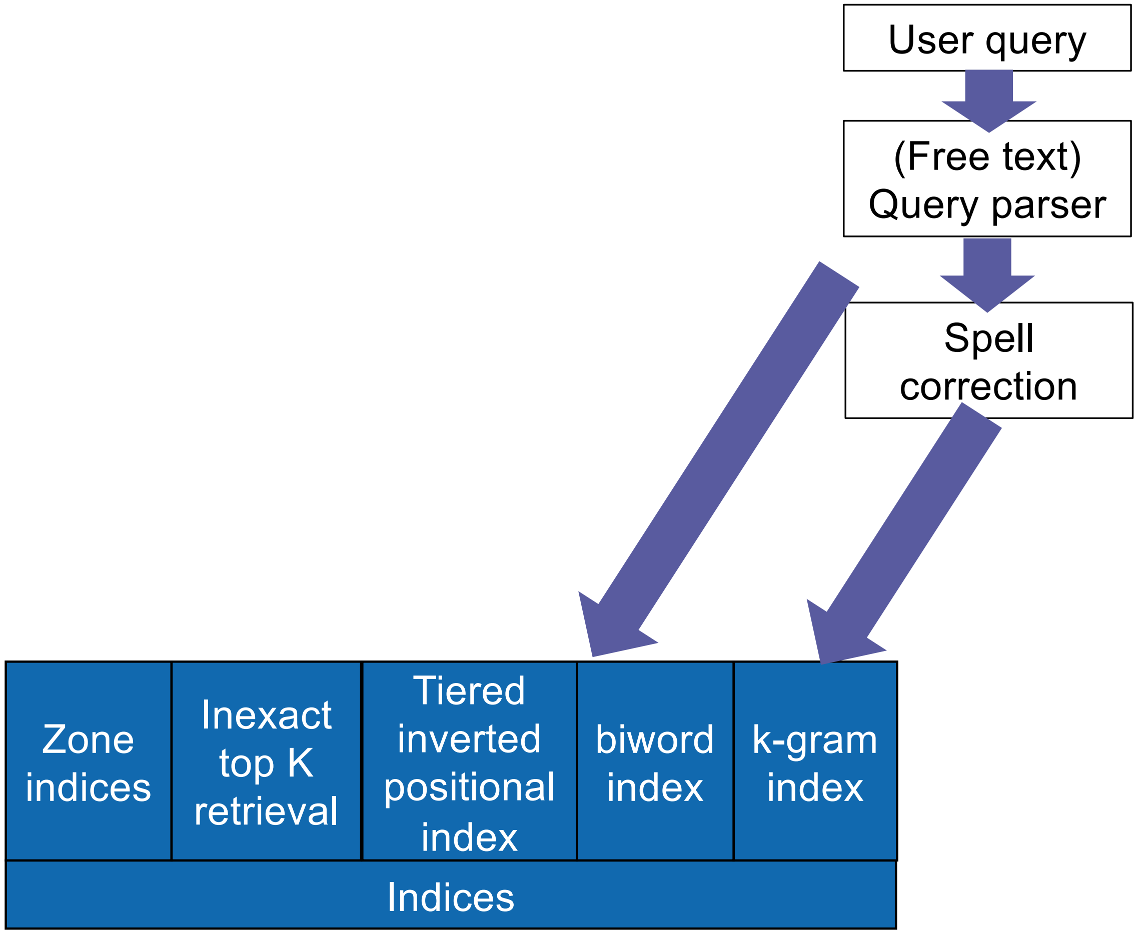
Input collection



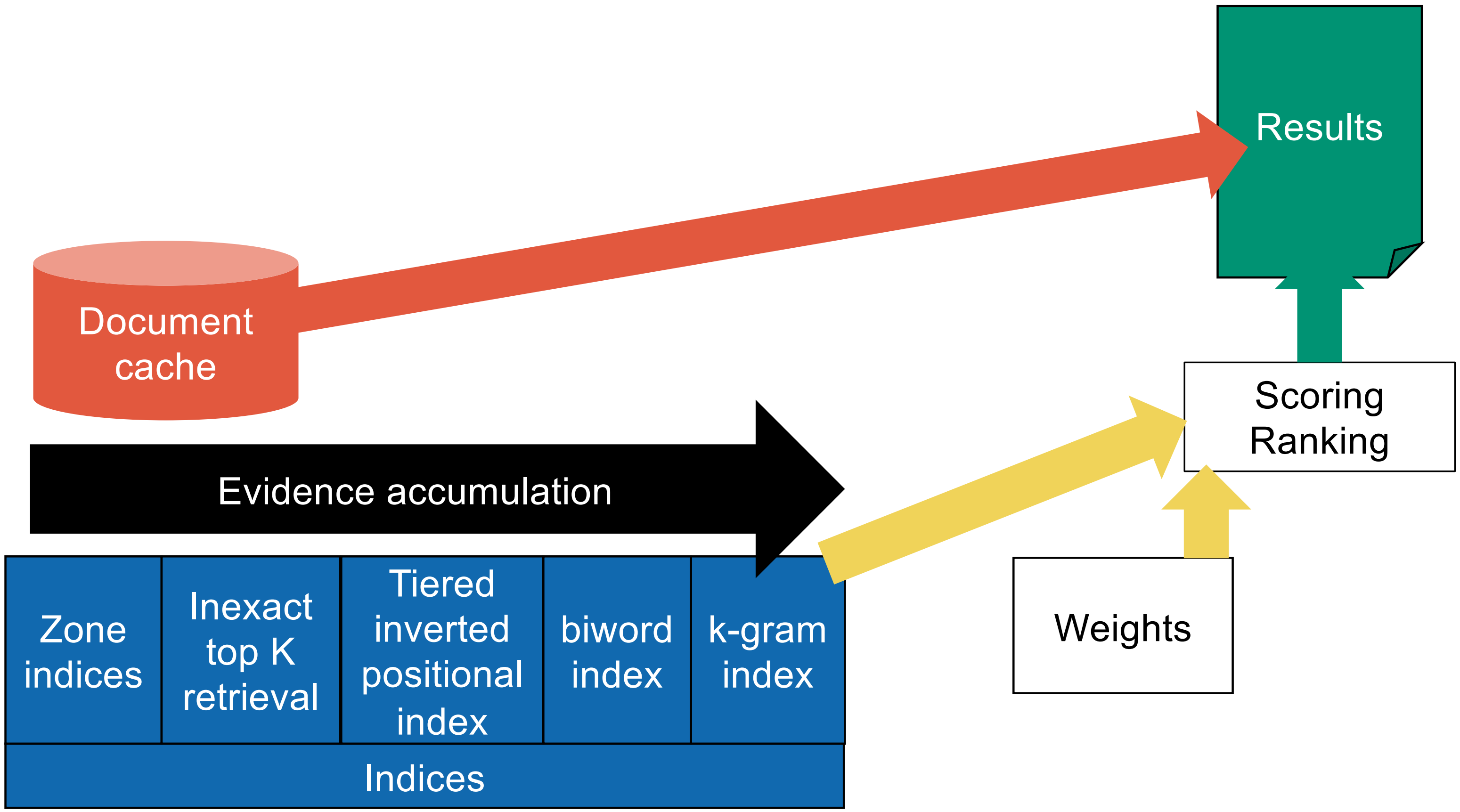
Indexing


Parsing
Linguistics
Indexers
Zone indices
Inexact top K retrieval
Tiered inverted positional index
biword index
k-gram index
Indices

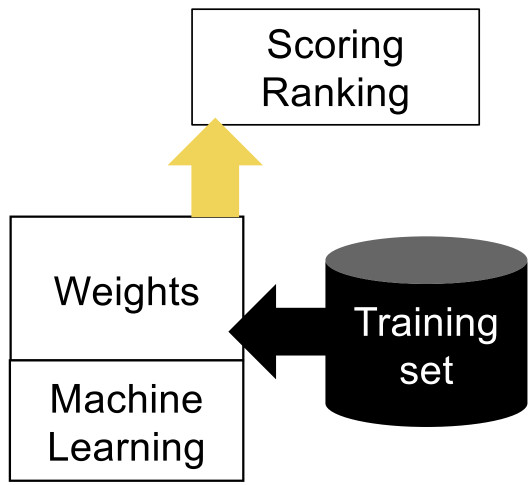
Querying


User query
(Free text)
Query parser
Spell
correction
Zone
indices
Inexact
top K
retrieval
Tiered
inverted
positional
index
biword
index
k-gram
index
Indices

Returning results



Fine tuning and training



Overall architecture